

Project Team

Abdelrahman Mohamed El-bahnsy

Ahmed Hamdy Mohamed Ahmed

Table of Contents

Project Planning & Management 7

1.1 Project Proposal 8

 1.1.1 Introduction: 8

 1.1.3 Problem Statement 8

 1.1.4 Project Objectives 9

 1.1.5 Project Scope..... 9

 1.1.6 Expected Outcomes..... 10

 1.2.1 Project Methodology..... 10

 1.2.6 Resource Allocation..... 13

 Hardware & Cloud Resources 14

1.3 Project Responsibilities 14

Literature Review 18

 2.1 Feedback & Evaluation 19

 2.1.1 Project Evaluation..... 19

 2.1.2 Lecturer's Feedback..... 19

2.2 Suggested Improvements 20

2.3 Final Grading Criteria 20

 3.2 User Stories 23

Chapter 4 28

System Analysis & Design 28

 4.1.1 Introduction 29

 4.1.2 Problem Statement 29

 4.1.3 Objectives..... 29

4.2 Use Case Diagram & Descriptions 30

 4.2.1 Overview 30

4.2.2 Actors..... 30

 1. Administrator 30

 2. Inventory Manager 30

 3. Monitoring System 31

4.2.3 Use Cases 31

4.2.4 Use Case Descriptions 31

UC-01 Login..... 31

UC-02 Add Product 32

UC-03 Update Product 32

UC-04 Delete Product 33

UC-05 Search Product 33

UC-06 View Dashboard 33

UC-07 Generate Reports 34

UC-08 Receive Low Stock Alert 34

UC-09 Monitor Services 34

UC-10 Deploy Application 35

4.3 Functional & Non-Functional Requirements 38

 4.3.1 Functional Requirements 38

 4.3.2 Non-Functional Requirements 38

 Performance..... 38

 Availability 38

 Scalability..... 39

 Security..... 39

 Reliability 39

 Maintainability 39

 Usability..... 39

 Compatibility 39

4.4 Software Architecture 40

 4.4.1 Architecture Overview 40

 4.4.2 Microservices Architecture 40

 4.4.3 Frontend 40

 4.4.4 Backend..... 40

 4.4.5 Database 41

 4.4.6 Monitoring 41

 4.4.7 CI/CD 41

 4.4.8 Kubernetes Architecture 42

 4.4.9 Architecture Diagram 42

4.5 Database Design & Data Modeling..... 43

 4.5.1 Database Overview 43

 4.5.2 ER Diagram..... 43

 43

 4.5.3 Tables..... 44

 4.5.4 Attributes 44

 Users..... 44

 Categories..... 44

 Products 44

 Inventory 45

 StockTransactions..... 45

 Alerts 45

 4.5.5 Relationships..... 45

 4.5.6 Primary Keys 45

 4.5.7 Foreign Keys..... 46

4.5.8 Normalization	46
4.7 UI/UX Design	49
4.7.1 Design Principles	49
4.7.2 Color Palette.....	49
4.7.3 Typography.....	50
4.7.4 Responsive Design.....	50
4.7.5 Wireframes.....	50
4.7.6 Mockups.....	51
4.7.7 Screenshots (Real System)	52
Figure 4.17 Dashboard.....	52
Figure 4.18 Orders Module.....	52
4.8 System Deployment & Integration.....	53
4.8.1 Technology Stack	53
4.8.2 Deployment Diagram	54
4.8.3 Component Diagram.....	55
Infrastructure Features.....	56
4.9.1 Authentication API	56
Example Request	56
Example Request	57
Example Response	57
4.9.3 Alert API	58
Example Request	58
Example Response	58
4.9.4 Metrics API.....	58
4.9.5 Request & Response Examples	58
Create Product	59
Update Inventory	59
Get Alerts.....	59
Retrieve Metrics	60
4.10 Testing & Validation	60
4.10.1 Unit Testing.....	60
4.10.2 Integration Testing.....	60
4.10.3 System Testing	61
4.10.4 User Acceptance Testing (UAT)	61
Table 4.X Testing Summary	61
Validation Summary	62
4.11 Deployment Strategy	62
4.11.1 Azure Infrastructure.....	62
4.11.2 Docker Containers.....	62

- 4.11.3 Kubernetes Deployment 63
- 4.11.4 Jenkins Pipeline 63
- 4.11.5 Prometheus Monitoring..... 63
- 4.11.6 Grafana Dashboards..... 64
- 4.11.7 Nginx Reverse Proxy..... 64
- 4.11.8 Scaling Strategy 64
- Deployment Workflow Summary 65
- Chapter 5 66
- Implementation (Source Code & Execution) 66
- 5.1 Source Code 67
- 5.1.1 Project Structure 67
- 5.1.2 Frontend Implementation..... 67
- 5.1.3 Backend Implementation 68
- 5.1.4 Database Implementation..... 68
- 5.1.5 Microservices Implementation 69
- 5.1.6 Monitoring Implementation 69
- 5.1.7 Infrastructure as Code..... 69
- 5.1.8 Containerization..... 69
- 5.1.9 Security & Error Handling..... 70
- 5.1.10 Coding Standards & Naming Conventions..... 70
- 5.2 Version Control & Collaboration 71
- 5.2.1 GitHub Repository..... 71
- 5.2.2 Repository Structure 71
- 5.2.3 Branching Strategy 72
- 5.2.4 Commit History 73
- 5.2.5 CI/CD Integration 73
- 5.3 Deployment & Execution 74
- 5.3.1 System Requirements 74
- Hardware Requirements 74
- Software Requirements..... 74
- 5.3.2 Installation Steps..... 74
- 5.3.3 Environment Configuration..... 75
- 5.3.4 Running the Application..... 75
- 5.3.5 Docker Deployment 75
- 5.3.6 Kubernetes Deployment 76
- 5.3.7 Jenkins Deployment Pipeline 76
- 5.3.8 Monitoring Execution..... 77
- 5.3.9 Deployment Verification 78
- 5.4 Source Code Highlights 78

5.4.1 Authentication Code 78

5.4.2 Inventory Management Code 79

5.4.3 Alert Service Code 79

5.4.4 Database Connection Code 80

5.4.5 Kubernetes Manifests 80

5.4.6 Jenkins Pipeline (Jenkinsfile) 81

5.4.7 Terraform Configuration 81

5.4.8 Ansible Playbooks 82

5.5 README & User Guide 82

5.5.1 Installation Guide 82

5.5.2 Configuration Guide 83

5.5.3 API Usage 83

5.5.4 Project Repository..... 83

5.5.5 Deployment Link 84

Testing & Quality Assurance 86

6.1 Test Cases & Test Plan..... 86

6.1.1 Testing Strategy..... 86

6.1.2 Testing Environment 86

6.1.3 Test Plan 86

6.1.4 Test Cases 87

6.1.5 Test Results 87

6.2 Automated Testing..... 87

6.2.1 API Testing..... 87

6.2.2 Kubernetes Deployment Validation 88

6.2.3 Monitoring Verification 88

6.2.4 CI/CD Pipeline Validation..... 89

6.3 Bug Reports..... 89

6.3.1 Identified Issues 90

6.3.2 Root Cause Analysis..... 90

6.3.3 Applied Fixes 90

6.3.4 Final Validation..... 90

Chapter 1

Project Planning & Management

1.1 Project Proposal

1.1.1 Introduction:

Inventory management is a critical component of modern business operations, directly affecting productivity, customer satisfaction, and operational costs. As organizations increasingly adopt cloud technologies and distributed applications, traditional inventory systems struggle to meet the requirements of scalability, automation, and high availability.

This project presents a **Cloud-Native Smart Inventory Management System** designed using a microservices architecture and modern DevOps practices. The system integrates Microsoft Azure, Docker, Kubernetes, Jenkins, Terraform, Ansible, Prometheus, and Grafana to provide an automated, scalable, and highly available platform for inventory management. By combining software development with DevOps methodologies, the project demonstrates how enterprise applications can be deployed, monitored, and maintained efficiently while ensuring reliability and continuous service availability.

1.1.2 Background:

Inventory management systems are essential for organizations that need to track products, monitor stock levels, and manage warehouse operations efficiently. However, many traditional systems rely on manual deployment, centralized architectures, and limited monitoring capabilities, making them difficult to scale and maintain.

Recent advances in cloud computing and DevOps have introduced technologies that simplify application deployment and infrastructure management. Containerization with Docker, orchestration through Kubernetes, Infrastructure as Code using Terraform, configuration management with Ansible, and Continuous Integration/Continuous Deployment (CI/CD) through Jenkins have become industry standards for building reliable software systems.

To take advantage of these technologies, this project adopts a cloud-native approach that combines modern software architecture with automated deployment and monitoring to deliver a reliable Smart Inventory platform.

1.1.3 Problem Statement

Many inventory management solutions suffer from limited scalability, manual deployment processes, and insufficient monitoring. As applications grow, maintaining a monolithic system becomes increasingly complex, resulting in longer deployment times, reduced flexibility, and increased operational costs.

Furthermore, the absence of real-time monitoring and automated alerting delays problem detection and recovery, affecting system availability and user experience. Manual infrastructure configuration also introduces inconsistencies between environments and increases the risk of deployment failures.

Therefore, there is a need for an inventory management system that supports scalability, automation, continuous deployment, and real-time observability while reducing operational complexity through modern DevOps practices.

1.1.4 Project Objectives

The primary objective of this project is to design and implement a cloud-native Smart Inventory Management System that demonstrates modern software engineering and DevOps principles.

The project aims to:

- Design a scalable microservices-based architecture.
- Develop responsive frontend and backend services for inventory management.
- Deploy the application on Microsoft Azure using Docker and Kubernetes.
- Automate infrastructure provisioning using Terraform.
- Automate server configuration using Ansible.
- Build a complete CI/CD pipeline using Jenkins.
- Implement centralized monitoring using Prometheus and Grafana.
- Configure automated alert notifications using Alertmanager.
- Improve deployment efficiency, system availability, and operational reliability.

1.1.5 Project Scope

This project covers the complete development and deployment lifecycle of a cloud-native inventory management platform. The system consists of multiple microservices responsible for inventory operations, backend processing, and alert management, supported by a web-based user interface.

The application is deployed on Microsoft Azure Virtual Machines using Docker containers orchestrated by Kubernetes. Infrastructure provisioning and configuration are automated using Terraform and Ansible, while Jenkins manages the CI/CD pipeline for automated building and deployment.

The monitoring layer collects infrastructure and application metrics using Prometheus and visualizes them through Grafana dashboards. Alertmanager is integrated to notify administrators of critical system events through email alerts.

Features such as multi-region deployment, disaster recovery, and advanced authentication mechanisms are considered beyond the scope of this project and are proposed as future enhancements.

1.1.6 Expected Outcomes

Upon completion, the project is expected to deliver a fully functional Smart Inventory Management System that reflects modern cloud-native application development and DevOps best practices.

The expected outcomes include:

- A scalable microservices-based inventory management platform.
- Automated application deployment through a Jenkins CI/CD pipeline.
- Containerized services managed by Kubernetes.
- Infrastructure provisioning using Terraform.
- Automated server configuration using Ansible.
- Reliable data storage using PostgreSQL.
- Real-time monitoring through Prometheus and Grafana dashboards.
- Automated email notifications for critical system alerts.
- Improved system availability, maintainability, and deployment efficiency.
- A production-oriented DevOps environment suitable for modern enterprise applications.

1.2 Project Plan

1.2.1 Project Methodology

The development of the Smart Inventory Management System followed the **Agile Software Development Methodology**, which supports iterative development, continuous feedback, and incremental delivery. Agile was selected because it allows the project to be developed in manageable phases while accommodating requirement changes and continuous improvements.

The project was divided into several iterations, starting with requirement analysis and system design, followed by application development, infrastructure automation, containerization, deployment, monitoring, and final testing. Each phase produced a working component that was validated before moving to the next stage.

This methodology ensured better collaboration among team members, faster issue resolution, and continuous integration of newly developed features throughout the project lifecycle.

1.2.2 Project Timeline

The project was completed over a period of **16 weeks**, with each phase focusing on a specific aspect of development and deployment. The timeline was organized to ensure a gradual progression from system planning to implementation, testing, and documentation.

Week	Activity
1-2	Requirements Gathering & Project Planning
3-4	System Analysis & Architecture Design
5-7	Frontend Development
5-8	Backend Microservices Development
8-9	Database Integration & API Testing
10-9	Docker Containerization
10-11	Kubernetes Deployment
11-12	Infrastructure Automation (Terraform & Ansible)
12-13	Jenkins CI/CD Pipeline
13-14	Monitoring with Prometheus & Grafana
15	System Testing & Optimization
16	Documentation & Final Presentation Preparation

1.2.4 Project Milestones

To monitor progress effectively, the project was divided into several key milestones.

Milestone	Description
M1	Project proposal approved
M2	System architecture completed
M3	Frontend implementation completed
M4	Backend microservices completed
M5	Database integration completed
M6	Docker containerization completed
M7	Kubernetes deployment completed
M8	CI/CD pipeline successfully implemented
M9	Monitoring and alerting configured
M10	Final testing completed
M11	Documentation and project delivery

These milestones provided measurable checkpoints that ensured the project remained aligned with the planned schedule and objectives.

1.2.5 Project Deliverables

At the end of the project, the following deliverables were successfully produced:

- Cloud-native Smart Inventory Management System.
- Responsive web-based frontend application.
- Backend microservices for inventory and alert management.
- PostgreSQL database with persistent storage.
- Docker images for all application services.
- Kubernetes deployment manifests.
- Jenkins CI/CD pipeline.
- Terraform infrastructure automation scripts.
- Ansible configuration management playbooks.
- Prometheus monitoring configuration.
- Grafana dashboards for infrastructure and application monitoring.
- Alertmanager email notification system.
- Complete technical documentation and user manual.

1.2.6 Resource Allocation

The project required both software and hardware resources to support development, deployment, and testing.

Software Resources

Resource	Purpose
Visual Studio Code	Source code development
Git & GitHub	Version control
Docker	Containerization
Kubernetes	Container orchestration
Jenkins	CI/CD automation
Terraform	Infrastructure provisioning
Ansible	Configuration management
Prometheus	Metrics collection
Grafana	Data visualization
PostgreSQL	Database management

Hardware & Cloud Resources

Resource	Purpose
Microsoft Azure Virtual Machine	Cloud infrastructure hosting
Linux (Ubuntu Server)	Application server
CPU, Memory & Storage	Hosting application containers
Internet Connection	Remote deployment and monitoring

The selected resources provided a complete environment for developing, deploying, automating, and monitoring the Smart Inventory Management System while reflecting modern enterprise DevOps practices.

1.3 Project Responsibilities

The Smart Inventory Management System was developed as an individual project, where all technical and engineering responsibilities were carried out by a single developer. This approach required managing the complete software development lifecycle, including system analysis, application development, cloud infrastructure, DevOps automation, deployment, monitoring, testing, and documentation.

Throughout the project, different responsibilities were performed sequentially while following modern software engineering and DevOps practices. This allowed complete control over the project architecture, implementation, deployment process, and operational monitoring.

The responsibilities undertaken during the project are summarized in the following table

Role	Responsibilities
Project Manager	Planned the project, defined objectives, managed the development schedule, monitored progress, and coordinated all project activities.
System Analyst	Collected and analyzed project requirements, identified system functionality, and designed the overall system architecture.
Frontend Developer	Developed the user interface using React, implemented responsive pages, integrated REST APIs, and enhanced the user experience.
Backend Developer	Developed Express.js microservices, implemented RESTful APIs, handled business logic, authentication, and communication between services.
Database Engineer	Designed and managed the PostgreSQL database, created the database schema, and maintained data integrity.
Cloud & DevOps Engineer	Provisioned Microsoft Azure infrastructure, implemented Docker containerization, Kubernetes orchestration, Jenkins CI/CD pipeline, Terraform infrastructure automation, Ansible configuration management, Prometheus monitoring, Grafana dashboards, and Alertmanager email notifications.
QA & Testing Engineer	Performed system testing, integration testing, deployment validation, debugging, and performance verification before production deployment.

As the sole developer, managing every phase of the project provided comprehensive experience across software engineering, cloud computing, DevOps, infrastructure automation, and system observability. This end-to-end involvement ensured consistency throughout the project and enabled the successful implementation of a production-oriented cloud-native application.

1.4 Risk Assessment & Mitigation Plan

Every software project involves potential risks that may affect development progress, deployment, or system reliability. Identifying these risks at an early stage helps minimize their impact and ensures the successful completion of the project. Throughout the development of the Smart Inventory Management System, several technical and operational risks were identified, and appropriate mitigation strategies were established to reduce their likelihood and impact.

The following table summarizes the major project risks and the corresponding mitigation plane

Risk	Probability	Impact	Mitigation Strategy
Azure Virtual Machine failure	Low	High	Regular VM monitoring, infrastructure backup, and rapid redeployment using Infrastructure as Code (Terraform).
Kubernetes deployment failure	Medium	High	Validate manifests before deployment, use rolling updates, and monitor pod health continuously.
Docker image build failure	Medium	Medium	Validate Dockerfiles, test images locally, and automate image building through Jenkins.
CI/CD pipeline failure	Medium	High	Perform pipeline validation, monitor Jenkins jobs, and review logs before deployment.
Database connectivity issues	Medium	High	Use persistent storage, secure database credentials, and monitor PostgreSQL availability.
Service downtime	Low	High	Configure Kubernetes self-healing, restart policies, and health probes to recover failed services automatically.
Monitoring or alert failure	Low	Medium	Continuously validate Prometheus targets, Alertmanager configuration, and Grafana dashboards.
Configuration errors	Medium	Medium	Manage infrastructure using Terraform and Ansible to maintain consistent environments.
Security vulnerabilities	Low	High	Store sensitive data in Kubernetes Secrets, restrict access permissions, and keep software components updated.
Project schedule delay	Medium	Medium	Follow Agile development practices, define milestones, and review project progress regularly.

The implementation of DevOps practices significantly reduced operational risks by introducing automation, continuous integration, infrastructure standardization, and proactive monitoring. These practices improved deployment reliability, minimized manual configuration errors, and enabled faster detection and resolution of system issues.

Overall, the adopted risk management strategy contributed to maintaining system stability, improving service availability, and ensuring the successful delivery of the Smart Inventory Management System.

1.5 Key Performance Indicators (KPIs)

Key Performance Indicators (KPIs) are measurable values used to evaluate the overall performance, reliability, and operational efficiency of the Smart Inventory Management System. These indicators provide continuous insights into application health, infrastructure stability, and service availability. During the project, Prometheus was used to collect metrics, while Grafana provided real-time visualization through interactive dashboards.

The primary KPIs used to evaluate the system are presented in Table 1.5

KPI	Target	Description
System Availability	$\geq 99\%$	Measures the percentage of time the application remains operational and accessible.
API Response Time	< 1 second	Measures the average response time of backend APIs to ensure fast request processing.
HTTP Success Rate	$\geq 99\%$	Measures the percentage of successful HTTP requests compared to total requests.
HTTP Error Rate (5xx)	$< 1\%$	Measures the percentage of server-side errors generated by the application.
Active Services	100% Running	Ensures that all microservices remain operational and healthy.
CI/CD Pipeline Success Rate	$\geq 95\%$	Measures the percentage of successful automated deployments executed by Jenkins.
Kubernetes Pod Health	All Pods Running	Verifies that all Kubernetes Pods remain in the Running state without failures.
Infrastructure Monitoring Coverage	100%	Ensures that all infrastructure components are continuously monitored by Prometheus.
Alert Notification Delivery	100%	Confirms that Alertmanager successfully delivers email notifications when alerts are triggered.
Database Availability	$\geq 99\%$	Measures the operational availability of the PostgreSQL database service.

These KPIs were continuously monitored throughout the project using Prometheus and Grafana dashboards. Whenever predefined thresholds were exceeded, Alertmanager automatically generated email notifications, enabling rapid identification and resolution of system issues. Monitoring these indicators helped maintain service reliability, improve deployment quality, and ensure the overall stability of the Smart Inventory Management System.

Chapter 2

Literature Review

2.1 Feedback & Evaluation

2.1.1 Project Evaluation

The Smart Inventory Management System was developed to address the limitations of traditional inventory management solutions by adopting a cloud-native architecture and modern DevOps practices. Throughout the project lifecycle, the system was continuously evaluated based on functionality, deployment automation, scalability, monitoring capabilities, and overall system reliability.

The implemented solution successfully integrates multiple technologies, including React for the frontend, Express.js microservices for backend services, PostgreSQL for data management, Docker for containerization, Kubernetes for orchestration, Jenkins for CI/CD automation, Terraform and Ansible for infrastructure management, and Prometheus with Grafana for monitoring and observability.

The evaluation confirmed that the project achieved its primary objectives by providing an automated deployment pipeline, reliable infrastructure, centralized monitoring, and real-time alerting. The use of microservices architecture also improved maintainability and simplified future scalability compared to traditional monolithic systems.

From a DevOps perspective, the project demonstrates a complete software delivery lifecycle, beginning with source code management and ending with automated deployment and continuous monitoring on Microsoft Azure infrastructure.

Overall, the implemented solution represents a practical application of modern software engineering principles and enterprise DevOps methodologies.

2.1.2 Lecturer's Feedback

During the project review process, the implemented solution was evaluated based on technical complexity, software quality, infrastructure automation, deployment methodology, monitoring capabilities, and documentation quality.

The project demonstrated several strengths, including:

- Adoption of a modern microservices architecture.
- Successful implementation of a complete CI/CD pipeline.
- Automated infrastructure provisioning and configuration management.
- Deployment on Microsoft Azure using Kubernetes.
- Comprehensive monitoring and alerting using Prometheus, Grafana, and Alertmanager.
- Well-organized project structure and documentation.

The evaluation also highlighted the successful integration of multiple DevOps tools into a single production-oriented environment, reflecting current industry practices.

2.2 Suggested Improvements

Although the Smart Inventory Management System successfully achieved its primary objectives, several enhancements can be implemented in future versions to improve scalability, security, and overall system performance.

One potential improvement is the deployment of the application across multiple availability zones or regions within Microsoft Azure. This would increase system resilience and provide better fault tolerance in case of infrastructure failures.

Security can also be enhanced by integrating a centralized authentication and authorization solution such as OAuth 2.0 or OpenID Connect, enabling secure user management and role-based access control. In addition, sensitive configuration data can be managed using dedicated secret management services such as Azure Key Vault instead of relying solely on Kubernetes Secrets.

From a monitoring perspective, distributed tracing tools such as Jaeger or OpenTelemetry can be introduced to provide end-to-end request tracing across microservices, making performance analysis and debugging more efficient.

The CI/CD pipeline may also be extended to include automated security scanning, code quality analysis, and vulnerability assessment before deployment. These additions would further improve software quality and reduce deployment risks.

Finally, implementing automatic horizontal scaling based on workload metrics would allow the application to dynamically allocate resources during periods of high demand, improving performance while optimizing infrastructure utilization.

By incorporating these enhancements, the system can evolve into a more robust, secure, and enterprise-ready cloud-native platform.

2.3 Final Grading Criteria

The project evaluation was based on several academic and technical aspects to ensure that both the implementation quality and the supporting documentation met the required standards. The assessment considered the overall project planning, software implementation, deployment process, testing activities, presentation quality, and technical documentation.

The grading criteria used for evaluating the project are summarized in the following table

Evaluation Criteria	Description	Weight (%)
Project Planning & Documentation	Quality of planning, analysis, documentation structure, and technical writing.	
System Design & Implementation	Architecture, software development, coding standards, and functionality.	
DevOps Implementation	Docker, Kubernetes, Jenkins, Terraform, Ansible, and deployment automation.	
Testing & Validation	Functional testing, integration testing, monitoring, and system verification.	
Presentation & Demonstration	Project presentation, technical explanation, live demonstration, and question handling.	
Total		100%

The grading process evaluates not only the final software product but also the engineering practices followed throughout the project lifecycle. Emphasis is placed on the adoption of modern DevOps methodologies, automation techniques, cloud deployment, system monitoring, and the ability to present and justify technical decisions effectively.

Chapter 3

Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder analysis is an essential step in the software development process, as it identifies the individuals and groups who interact with the system or are affected by its operation. Understanding stakeholder needs ensures that the developed solution satisfies both functional and operational requirements.

For the Smart Inventory Management System, the primary stakeholders include the system administrator, warehouse staff, inventory manager, and DevOps engineer. Each stakeholder has different responsibilities and expectations that influence the overall system design.

The following table summarizes the main stakeholders and their responsibilities

Stakeholder	Responsibilities	System Needs
System Administrator	Manages users, monitors system health, and oversees deployments.	Secure access, monitoring dashboards, and system management tools.
Inventory Manager	Monitors inventory levels and manages products.	Accurate inventory information and reporting.
Warehouse Staff	Performs inventory operations such as adding, updating, and tracking products.	Simple and responsive user interface with fast system response.
DevOps Engineer	Deploys, monitors, and maintains the infrastructure and application services.	CI/CD pipeline, monitoring dashboards, logging, and alert notifications.

Understanding these stakeholders helped define the system requirements and ensured that the developed platform addresses both operational and technical needs.

3.2 User Stories

User stories describe the system requirements from the user's perspective and help ensure that the application delivers value to its intended users.

The primary user stories are listed below

ID	User Story
US-01	As an Administrator, I want to monitor all system services so that I can quickly detect failures.
US-02	As an Inventory Manager, I want to add new products so that inventory remains up to date.
US-03	As an Inventory Manager, I want to edit product information whenever changes occur.
US-04	As an Inventory Manager, I want to remove discontinued products from inventory.
US-05	As a Warehouse Staff member, I want to search for products quickly.
US-06	As a Warehouse Staff member, I want to update stock quantities after receiving shipments.
US-07	As a DevOps Engineer, I want deployments to be automated through Jenkins.
US-08	As a DevOps Engineer, I want Prometheus and Grafana to monitor all services continuously.
US-09	As an Administrator, I want to receive email alerts whenever a critical failure occurs.

UC-01 Login

Item	Description
Use Case ID	UC-01
Use Case Name	Login
Actor	Administrator
Description	User logs into the system.
Preconditions	User account exists.
Main Flow	Enter credentials → Validate → Access Dashboard.
Alternative Flow	Invalid credentials.
Postconditions	User is authenticated successfully.

Functional Requirements

ID	Requirement
FR-01	The system shall allow users to authenticate securely.
FR-02	The system shall allow product creation.
FR-03	The system shall allow product updates.
FR-04	The system shall allow product deletion.
FR-05	The system shall display inventory information.
FR-06	The system shall expose REST APIs.
FR-07	The system shall support Docker deployment.
FR-08	The system shall support Kubernetes orchestration.
FR-09	The system shall support automated CI/CD deployment.
FR-10	The system shall provide monitoring dashboards.
FR-11	The system shall send email alerts on critical failures.

Non-Functional Requirements

Category	Requirement
Performance	API response time should be less than 1 second.
Availability	System availability should exceed 99%.
Scalability	Services should scale independently using Kubernetes.
Reliability	Failed containers should restart automatically.
Security	Sensitive information shall be stored securely.
Maintainability	Infrastructure shall be managed using Infrastructure as Code.
Monitoring	All services shall be monitored continuously.
Usability	The user interface shall be simple and responsive.

1) STAKEHOLDER DIAGRAM

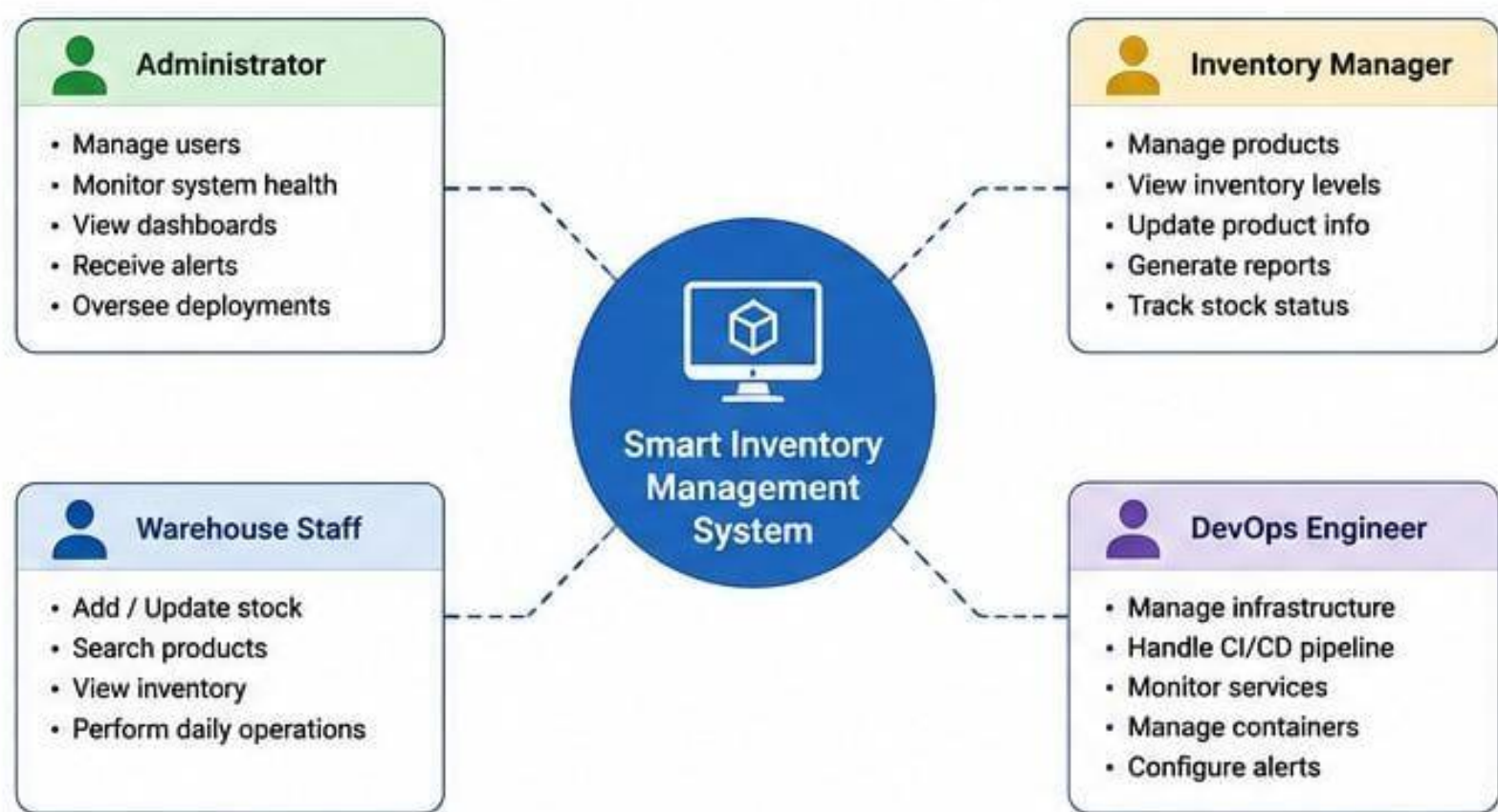


Figure 3.1: Stakeholder Diagram

3) USE CASE FLOW – ADD PRODUCT (EXAMPLE)



Figure 3.3: Use Case Flow – Add Product

4) USE CASE DESCRIPTION – UC-02: ADD PRODUCT

Item	Description
Use Case ID	UC-02
Use Case Name	Add Product
Actor	Inventory Manager
Description	Allows the inventory manager to add a new product to the system.
Preconditions	User must be logged in and have appropriate permissions.
Main Flow	1. User navigates to Products. 2. User clicks Add Product. 3. User enters product details (name, category, quantity, price). 4. System validates the data. 5. System saves the product to the database. 6. System displays success message.
Alternative Flow	If data is invalid, system shows error message and asks for correct information.
Postconditions	New product is added to the inventory successfully.

Figure 3.4: Use Case Description Example

5) REQUIREMENTS CLASSIFICATION DIAGRAM

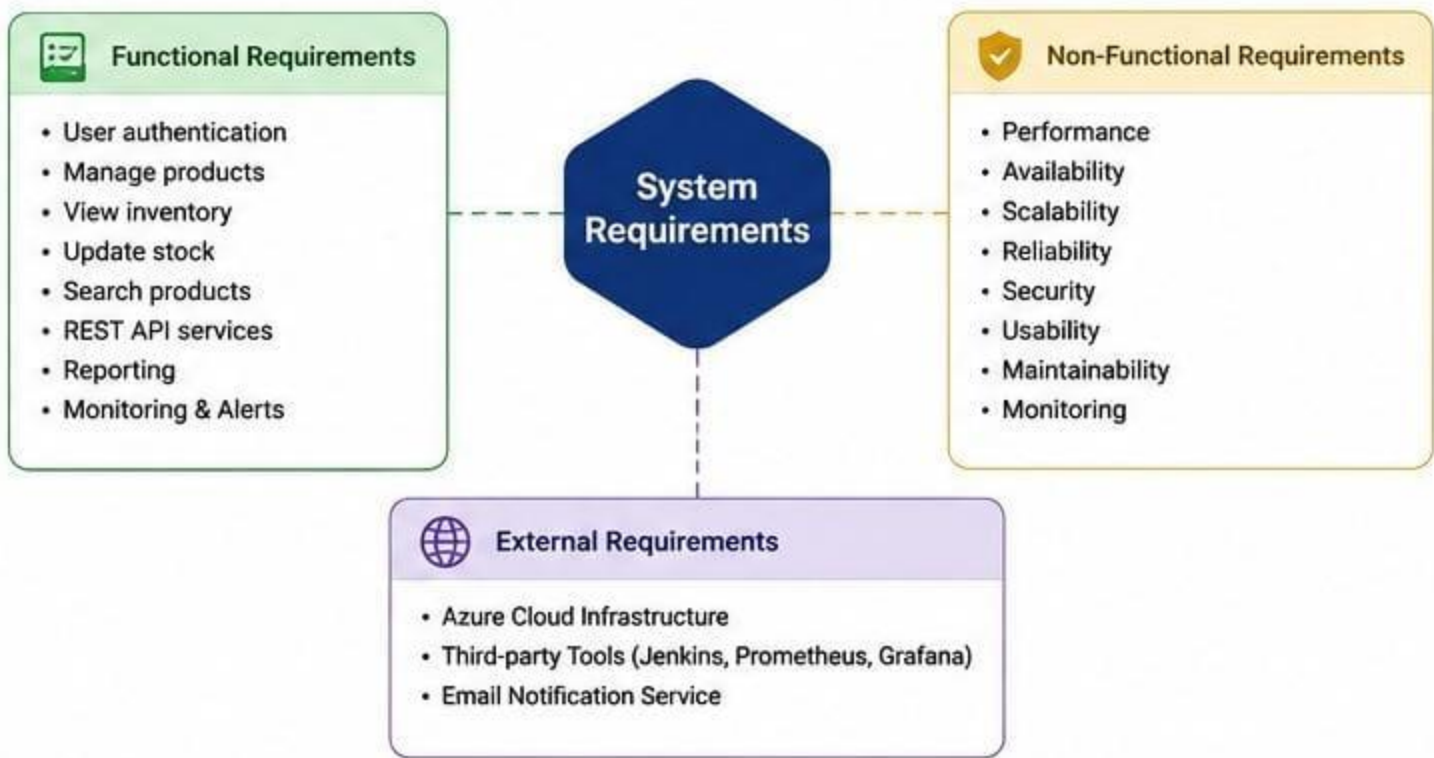


Figure 3.5: Requirements Classification Diagram

2) USE CASE DIAGRAM (UML)

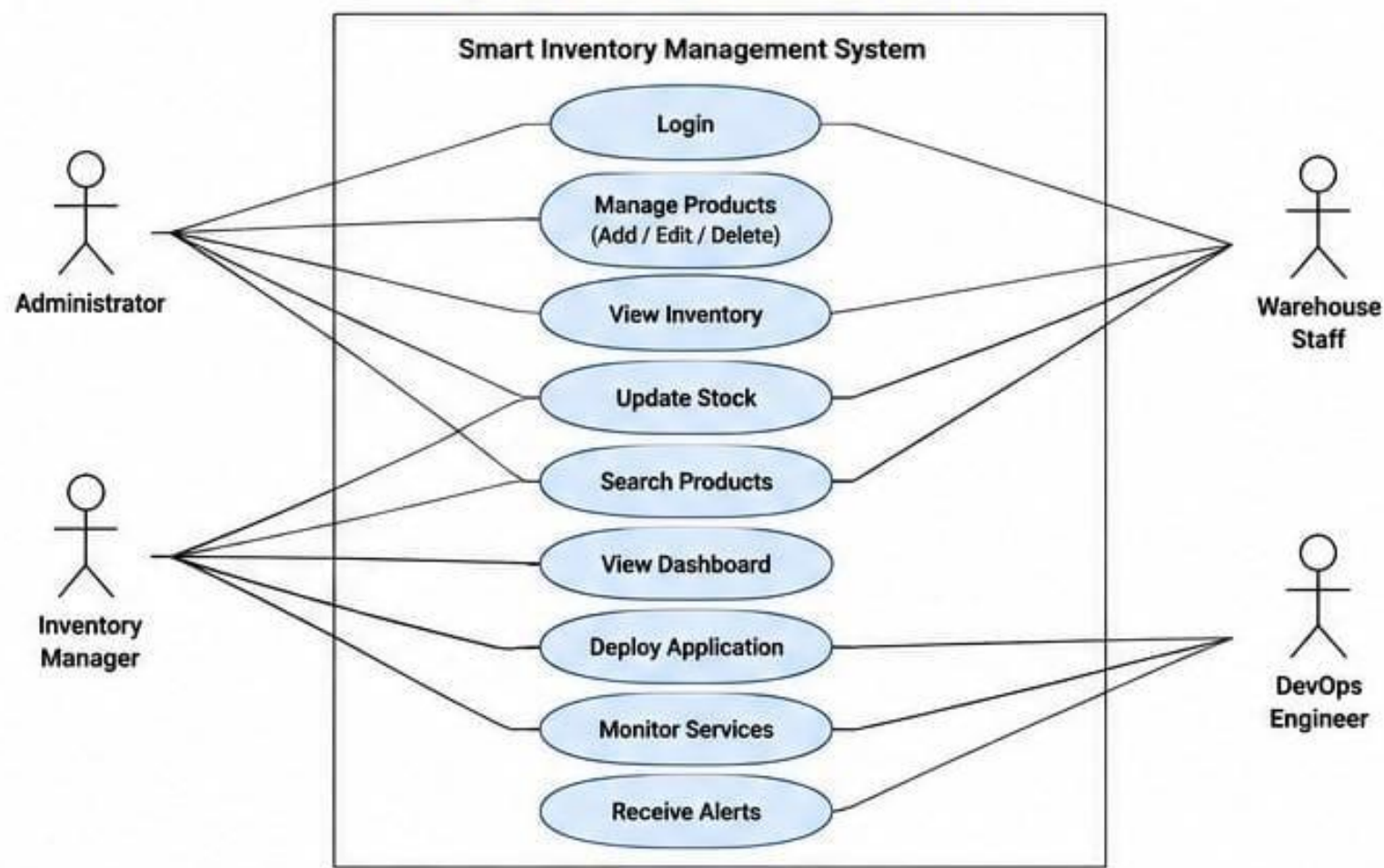


Figure 3.2: Use Case Diagram

Chapter 4

System Analysis & Design

4.1 Problem Statement & Objectives

4.1.1 Introduction

The Smart Inventory Management System is a cloud-native inventory management platform designed to help organizations monitor and manage inventory efficiently in real time. The system follows a microservices architecture and integrates modern DevOps technologies, including Docker, Kubernetes, Jenkins, Terraform, Ansible, Prometheus, Grafana, and Microsoft Azure.

The platform provides inventory management, stock monitoring, automated alert generation, and centralized monitoring dashboards. It also incorporates CI/CD pipelines and Infrastructure as Code (IaC) to automate deployment, improve scalability, and simplify system maintenance.

4.1.2 Problem Statement

Many organizations continue to rely on traditional inventory management methods or monolithic systems that lack scalability, automation, and real-time monitoring. These limitations often result in delayed stock updates, inefficient resource utilization, slow deployments, and increased operational costs.

Furthermore, managing distributed services without proper monitoring makes it difficult to detect system failures, analyze performance, or maintain high service availability. Manual deployment processes also increase the risk of configuration errors and downtime.

The project addresses these challenges by developing a cloud-native inventory management system that combines microservices architecture with DevOps practices to provide automated deployment, centralized monitoring, improved scalability, and reliable inventory operations.

4.1.3 Objectives

The primary objectives of this project are:

- Develop a scalable Smart Inventory Management System using a microservices architecture.
- Containerize all services using Docker for consistent and portable deployments.
- Deploy the application on Kubernetes to achieve scalability and high availability.
- Automate infrastructure provisioning using Terraform.
- Automate server configuration and application deployment using Ansible.
- Implement a CI/CD pipeline using Jenkins for continuous integration and deployment.
- Monitor system health, application performance, and infrastructure metrics using Prometheus and Grafana.
- Provide real-time inventory tracking and stock management.
- Generate automated alerts for inventory and system events.
- Host the solution on Microsoft Azure cloud infrastructure.
- Improve system reliability, maintainability, and deployment efficiency through DevOps best practices.

- Demonstrate the integration of modern cloud-native technologies within a complete production-ready environment.

4.2 Use Case Diagram & Descriptions

4.2.1 Overview

A Use Case Diagram illustrates the interaction between the users and the Smart Inventory Management System. It identifies the main actors of the system and the services provided by the application. This diagram helps define the functional requirements from the user's perspective and clarifies how each actor interacts with the system.

The Smart Inventory Management System includes three primary actors: the Administrator, the Inventory Manager, and the Monitoring System. Each actor performs a specific set of operations to ensure efficient inventory management, automated monitoring, and system administration.

4.2.2 Actors

1. Administrator

The Administrator is responsible for managing the entire system, including user management, service deployment, monitoring infrastructure, and overall system configuration.

Responsibilities

- Login to the system
- Manage inventory records
- Manage users
- View monitoring dashboards
- Configure alerts
- Monitor application health
- Deploy application updates

2. Inventory Manager

The Inventory Manager is responsible for managing products and monitoring stock availability.

Responsibilities

- Login
 - Add products
 - Update products
 - Delete products
 - View inventory
 - Monitor stock levels
 - Receive low-stock alerts
 - Generate inventory reports
-

3. Monitoring System

The Monitoring System represents the automated monitoring tools integrated into the project, including Prometheus, Grafana, and Alert Manager.

Responsibilities

- Collect application metrics
- Monitor container health
- Detect service failures
- Generate alerts
- Display monitoring dashboards

4.2.3 Use Cases

The Smart Inventory Management System provides the following primary use cases

Use Case	Description	Actor
Login	Authenticate users before accessing the system	Administrator, Inventory Manager
Manage Inventory	Add, update, delete, and search inventory items	Inventory Manager
View Inventory	Display available products and quantities	Inventory Manager
Generate Reports	Generate inventory statistics and reports	Inventory Manager
Receive Alerts	Notify users when stock becomes low	Inventory Manager
Monitor Services	Monitor application health and performance	Monitoring System
View Dashboards	Display Grafana dashboards and metrics	Administrator
Deploy Application	Deploy new application versions through CI/CD	Administrator
Configure Infrastructure	Provision infrastructure using Terraform and Ansible	Administrator
Monitor Kubernetes Cluster	Track Kubernetes Pods and Services	Monitoring System

4.2.4 Use Case Descriptions

UC-01 Login

Primary Actor: Administrator, Inventory Manager

Description:

Allows authorized users to securely log into the system.

Preconditions

- User account exists.
- User is registered in the system.

Main Flow

1. User enters username and password.
2. System validates credentials.
3. User is redirected to the dashboard.

Postconditions

- User is successfully authenticated.

UC-02 Add Product

Primary Actor: Inventory Manager

Description:

Allows the inventory manager to add a new product to the inventory database.

Preconditions

- User is logged in.

Main Flow

1. Select Add Product.
2. Enter product information.
3. Save product.

Postconditions

- Product is stored successfully.

UC-03 Update Product

Primary Actor: Inventory Manager

Description:

Allows updating existing product information.

Preconditions

- Product already exists.

Main Flow

1. Search product.
2. Edit information.
3. Save changes.

Postconditions

- Product information is updated.

UC-04 Delete Product

Primary Actor: Inventory Manager

Description:

Allows removing products from inventory.

Preconditions

- Product exists.

Main Flow

1. Select product.
2. Click Delete.
3. Confirm deletion.

Postconditions

- Product is removed.

UC-05 Search Product

Primary Actor: Inventory Manager

Description:

Allows searching products using name or ID.

Preconditions

- User logged in.

Main Flow

1. Enter search keyword.
2. System retrieves matching products.

Postconditions

- Matching products are displayed.

UC-06 View Dashboard

Primary Actor: Administrator

Description:

Displays real-time monitoring dashboards.

Preconditions

- Administrator logged in.

Main Flow

1. Open Dashboard.
2. System loads monitoring metrics.

Postconditions

- Dashboard is displayed.

UC-07 Generate Reports

Primary Actor: Inventory Manager

Description:

Generates inventory reports.

Preconditions

- Inventory data available.

Main Flow

1. Select report type.
2. Generate report.
3. View report.

Postconditions

- Report generated successfully.

UC-08 Receive Low Stock Alert

Primary Actor: Inventory Manager

Description:

Notifies managers when stock falls below the minimum threshold.

Preconditions

- Product quantity reaches threshold.

Main Flow

1. System detects low stock.
2. Alert Service generates notification.
3. Manager receives alert.

Postconditions

- Alert delivered successfully.

UC-09 Monitor Services

Primary Actor: Monitoring System

Description:

Continuously monitors application microservices using Prometheus.

Preconditions

- Services are running.

Main Flow

- 1. Prometheus scrapes metrics.
- 2. Metrics are stored.
- 3. Grafana displays results.

Postconditions

- Service status is updated.

UC-10 Deploy Application

Primary Actor: Administrator

Description:

Deploys the complete application to Kubernetes using the CI/CD pipeline.

Preconditions

- Source code is available.
- Kubernetes cluster is running.

Main Flow

- 1. Push code to GitHub.
- 2. Jenkins starts pipeline.
- 3. Docker images are built.
- 4. Kubernetes updates deployment.

Postconditions

- New application version is deployed successfully.

USE CASE DESCRIPTIONS – SMART INVENTORY MANAGEMENT SYSTEM

UC-01Login  Primary Actor Administrator, Inventory Manager Description Allows authorized users to securely log into the system. Preconditions • User account exists. • User is registered in the system. Main Flow 1. User enters username and password. 2. System validates credentials. 3. User is redirected to the dashboard. Postconditions • User is successfully authenticated. <td> UC-02Add Product  Primary Actor Inventory Manager Description Allows the inventory manager to add a new product to the inventory database. Preconditions • User is logged in. Main Flow 1. Select Add Product. 2. Enter product information. 3. Save product. Postconditions • Product is stored successfully. <td> UC-03Update Product  Primary Actor Inventory Manager Description Allows updating existing product information. Preconditions • Product already exists. Main Flow 1. Search product. 2. Edit information. 3. Save changes. Postconditions • Product information is updated. <td> UC-04Delete Product  Primary Actor Inventory Manager Description Allows removing products from inventory. Preconditions • Product exists. Main Flow 1. Select product. 2. Click Delete. 3. Confirm deletion. Postconditions • Product is removed successfully. <td> UC-05Search Product  Primary Actor Inventory Manager Description Allows searching products using name or ID. Preconditions • User is logged in. Main Flow 1. Enter search keyword. 2. System retrieves matching products. Postconditions • Matching products are displayed. </td></td></td></td>	UC-02Add Product  Primary Actor Inventory Manager Description Allows the inventory manager to add a new product to the inventory database. Preconditions • User is logged in. Main Flow 1. Select Add Product. 2. Enter product information. 3. Save product. Postconditions • Product is stored successfully. <td> UC-03Update Product  Primary Actor Inventory Manager Description Allows updating existing product information. Preconditions • Product already exists. Main Flow 1. Search product. 2. Edit information. 3. Save changes. Postconditions • Product information is updated. <td> UC-04Delete Product  Primary Actor Inventory Manager Description Allows removing products from inventory. Preconditions • Product exists. Main Flow 1. Select product. 2. Click Delete. 3. Confirm deletion. Postconditions • Product is removed successfully. <td> UC-05Search Product  Primary Actor Inventory Manager Description Allows searching products using name or ID. Preconditions • User is logged in. Main Flow 1. Enter search keyword. 2. System retrieves matching products. Postconditions • Matching products are displayed. </td></td></td>	UC-03Update Product  Primary Actor Inventory Manager Description Allows updating existing product information. Preconditions • Product already exists. Main Flow 1. Search product. 2. Edit information. 3. Save changes. Postconditions • Product information is updated. <td> UC-04Delete Product  Primary Actor Inventory Manager Description Allows removing products from inventory. Preconditions • Product exists. Main Flow 1. Select product. 2. Click Delete. 3. Confirm deletion. Postconditions • Product is removed successfully. <td> UC-05Search Product  Primary Actor Inventory Manager Description Allows searching products using name or ID. Preconditions • User is logged in. Main Flow 1. Enter search keyword. 2. System retrieves matching products. Postconditions • Matching products are displayed. </td></td>	UC-04Delete Product  Primary Actor Inventory Manager Description Allows removing products from inventory. Preconditions • Product exists. Main Flow 1. Select product. 2. Click Delete. 3. Confirm deletion. Postconditions • Product is removed successfully. <td> UC-05Search Product  Primary Actor Inventory Manager Description Allows searching products using name or ID. Preconditions • User is logged in. Main Flow 1. Enter search keyword. 2. System retrieves matching products. Postconditions • Matching products are displayed. </td>	UC-05Search Product  Primary Actor Inventory Manager Description Allows searching products using name or ID. Preconditions • User is logged in. Main Flow 1. Enter search keyword. 2. System retrieves matching products. Postconditions • Matching products are displayed.
UC-06View Dashboard  Primary Actor Administrator Description Displays real-time monitoring dashboards and key metrics. Preconditions • Administrator is logged in. Main Flow 1. Open Dashboard. 2. System loads monitoring metrics. 3. Dashboard is displayed. Postconditions • Dashboard is displayed successfully. <td> UC-07Generate Reports  Primary Actor Inventory Manager Description Generates inventory reports based on selected criteria. Preconditions • Inventory data is available. Main Flow 1. Select report type. 2. Generate report. 3. View or download report. Postconditions • Report is generated successfully. <td> UC-08Receive Low Stock Alert  Primary Actor Inventory Manager Description Notifies managers when stock falls below the minimum threshold. Preconditions • Product quantity reaches the predefined threshold. Main Flow 1. System detects low stock. 2. Alert Service generates notification. 3. Manager receives alert. Postconditions • Alert is delivered successfully. <td> UC-09Monitor Services  Primary Actor Monitoring System Description Continuously monitors application microservices using Prometheus. Preconditions • Services are running. Main Flow 1. Prometheus scrapes metrics. 2. Metrics are stored. 3. Grafana displays results. Postconditions • Service status is updated. <td> UC-10Deploy Application  Primary Actor Administrator Description Deploys the complete application to Kubernetes using the CI/CD pipeline. Preconditions • Source code is available. • Kubernetes cluster is running. Main Flow 1. Push code to GitHub. 2. Jenkins starts pipeline. 3. Docker images are built. 4. Kubernetes updates deployment. Postconditions • New application version is deployed successfully. </td></td></td></td>	UC-07Generate Reports  Primary Actor Inventory Manager Description Generates inventory reports based on selected criteria. Preconditions • Inventory data is available. Main Flow 1. Select report type. 2. Generate report. 3. View or download report. Postconditions • Report is generated successfully. <td> UC-08Receive Low Stock Alert  Primary Actor Inventory Manager Description Notifies managers when stock falls below the minimum threshold. Preconditions • Product quantity reaches the predefined threshold. Main Flow 1. System detects low stock. 2. Alert Service generates notification. 3. Manager receives alert. Postconditions • Alert is delivered successfully. <td> UC-09Monitor Services  Primary Actor Monitoring System Description Continuously monitors application microservices using Prometheus. Preconditions • Services are running. Main Flow 1. Prometheus scrapes metrics. 2. Metrics are stored. 3. Grafana displays results. Postconditions • Service status is updated. <td> UC-10Deploy Application  Primary Actor Administrator Description Deploys the complete application to Kubernetes using the CI/CD pipeline. Preconditions • Source code is available. • Kubernetes cluster is running. Main Flow 1. Push code to GitHub. 2. Jenkins starts pipeline. 3. Docker images are built. 4. Kubernetes updates deployment. Postconditions • New application version is deployed successfully. </td></td></td>	UC-08Receive Low Stock Alert  Primary Actor Inventory Manager Description Notifies managers when stock falls below the minimum threshold. Preconditions • Product quantity reaches the predefined threshold. Main Flow 1. System detects low stock. 2. Alert Service generates notification. 3. Manager receives alert. Postconditions • Alert is delivered successfully. <td> UC-09Monitor Services  Primary Actor Monitoring System Description Continuously monitors application microservices using Prometheus. Preconditions • Services are running. Main Flow 1. Prometheus scrapes metrics. 2. Metrics are stored. 3. Grafana displays results. Postconditions • Service status is updated. <td> UC-10Deploy Application  Primary Actor Administrator Description Deploys the complete application to Kubernetes using the CI/CD pipeline. Preconditions • Source code is available. • Kubernetes cluster is running. Main Flow 1. Push code to GitHub. 2. Jenkins starts pipeline. 3. Docker images are built. 4. Kubernetes updates deployment. Postconditions • New application version is deployed successfully. </td></td>	UC-09Monitor Services  Primary Actor Monitoring System Description Continuously monitors application microservices using Prometheus. Preconditions • Services are running. Main Flow 1. Prometheus scrapes metrics. 2. Metrics are stored. 3. Grafana displays results. Postconditions • Service status is updated. <td> UC-10Deploy Application  Primary Actor Administrator Description Deploys the complete application to Kubernetes using the CI/CD pipeline. Preconditions • Source code is available. • Kubernetes cluster is running. Main Flow 1. Push code to GitHub. 2. Jenkins starts pipeline. 3. Docker images are built. 4. Kubernetes updates deployment. Postconditions • New application version is deployed successfully. </td>	UC-10Deploy Application  Primary Actor Administrator Description Deploys the complete application to Kubernetes using the CI/CD pipeline. Preconditions • Source code is available. • Kubernetes cluster is running. Main Flow 1. Push code to GitHub. 2. Jenkins starts pipeline. 3. Docker images are built. 4. Kubernetes updates deployment. Postconditions • New application version is deployed successfully.

Figure 4.2.1: Use Case Diagram – Smart Inventory Management System

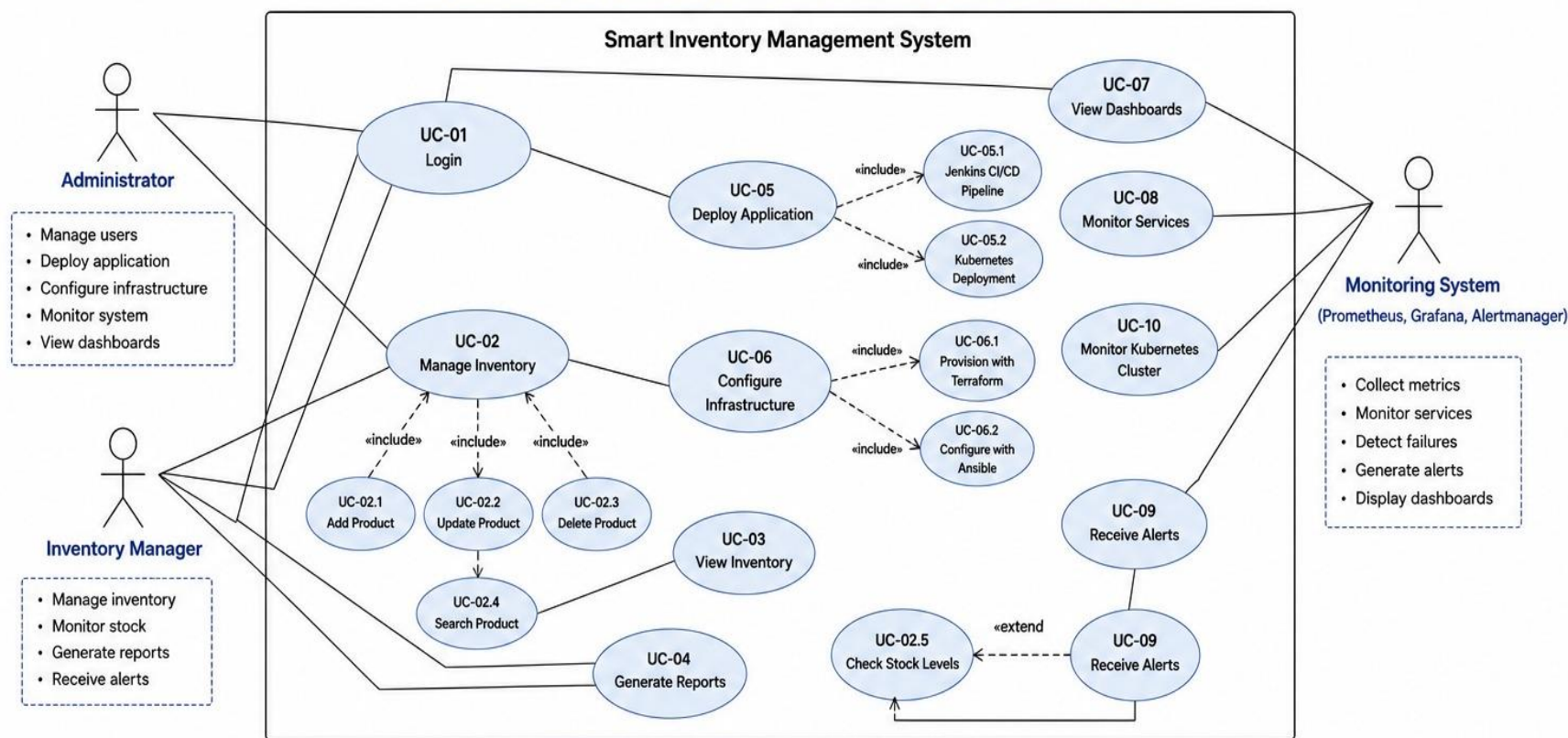
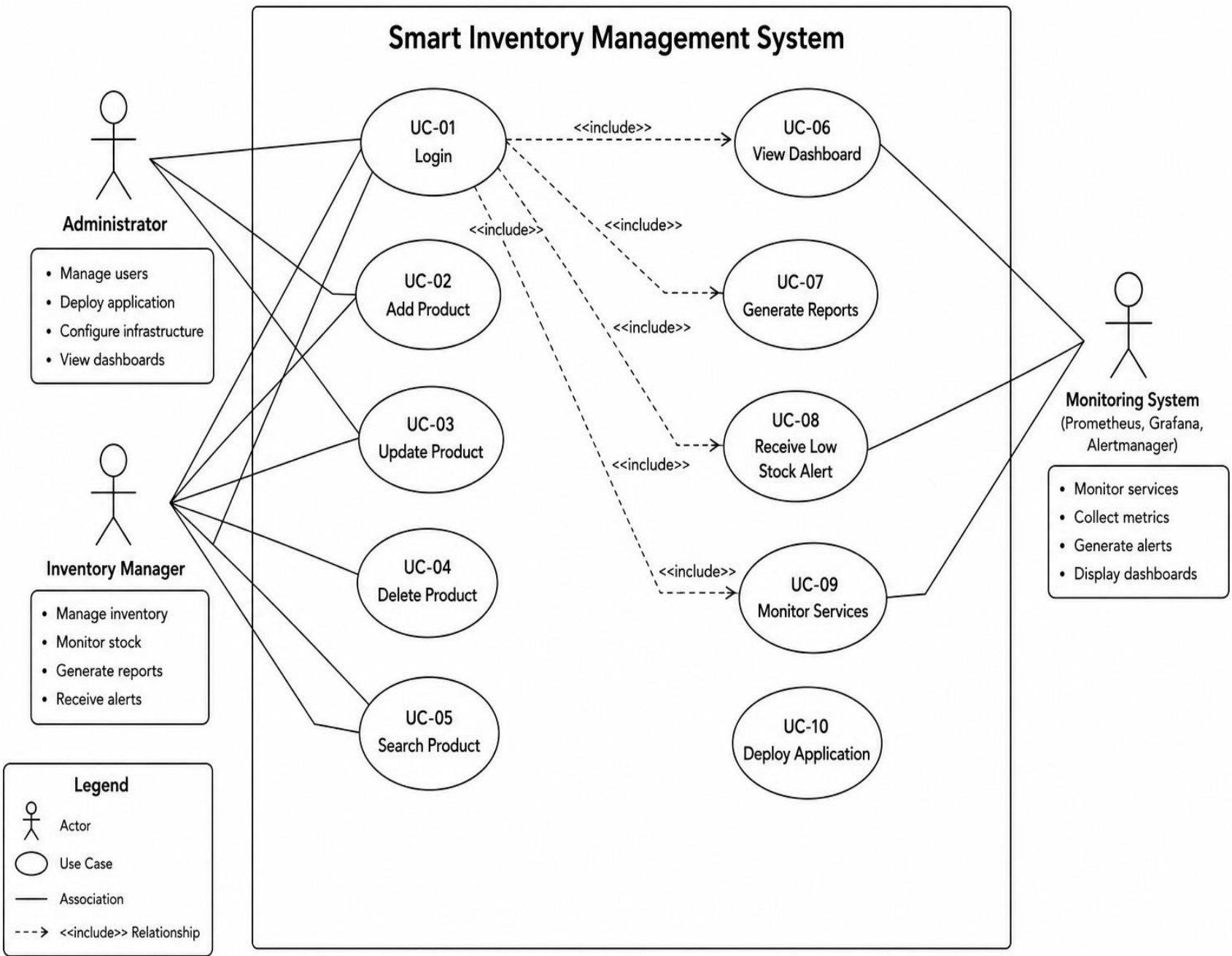
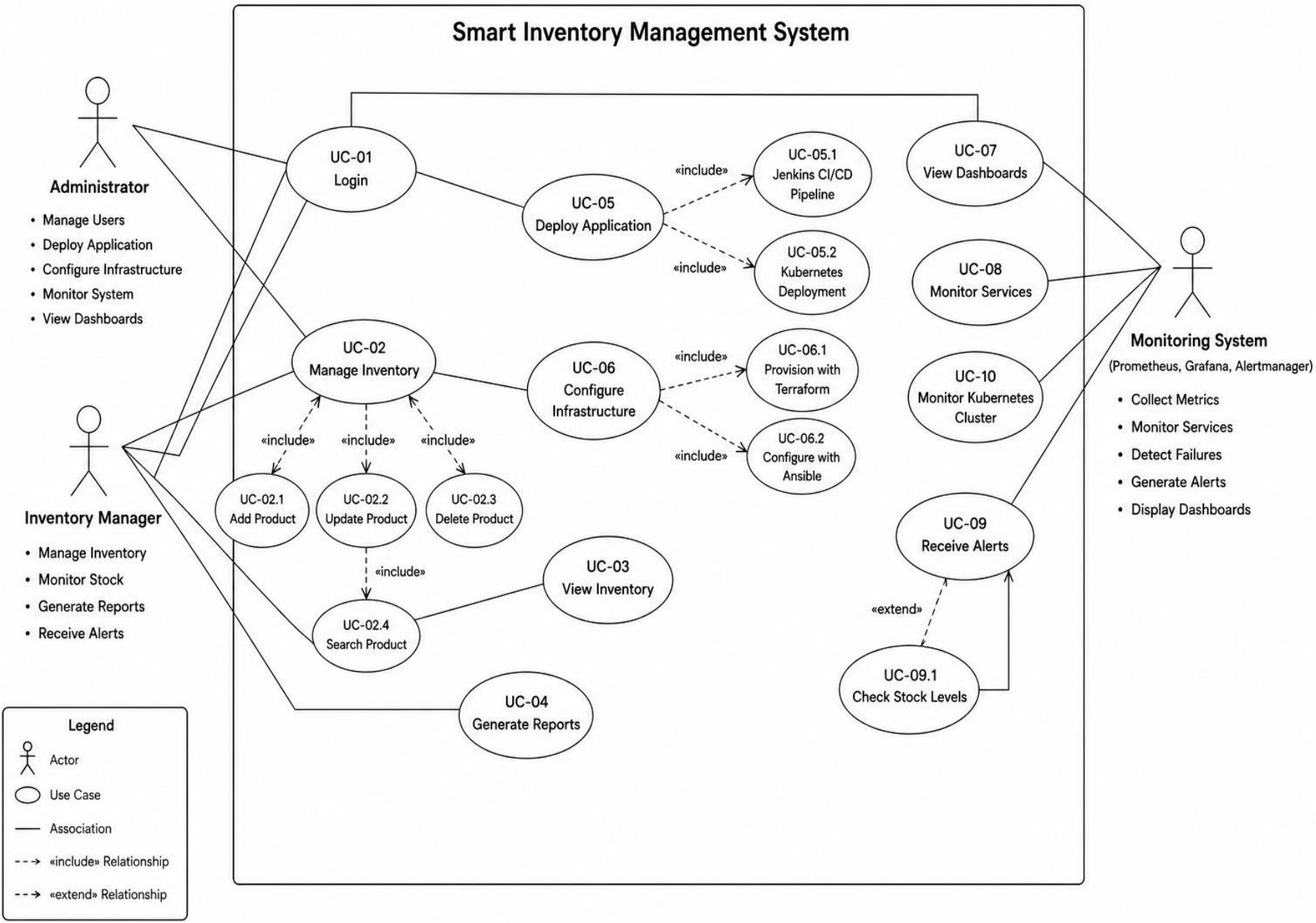


Table 4.2.1: Use Case Descriptions

Use Case ID	Use Case Name	Actor(s)	Description
UC-01	Login	Administrator, Inventory Manager	Authenticate users before accessing the system.
UC-02	Manage Inventory	Inventory Manager	Add, update, delete, and search inventory items.
UC-03	View Inventory	Inventory Manager	View list of products and their current stock levels.
UC-04	Generate Reports	Inventory Manager	Generate inventory reports and statistics.
UC-05	Deploy Application	Administrator	Deploy new application versions using CI/CD pipeline.
UC-06	Configure Infrastructure	Administrator	Provision and configure infrastructure using Terraform and Ansible.
UC-07	View Dashboards	Administrator	View Grafana dashboards and system metrics.
UC-08	Monitor Services	Monitoring System	Collect and monitor application and infrastructure metrics.
UC-09	Receive Alerts	Inventory Manager, Administrator	Receive alerts for low stock or system failures.
UC-10	Monitor Kubernetes Cluster	Monitoring System	Monitor Kubernetes pods, services, and cluster health.



4.3 Functional & Non-Functional Requirements

4.3.1 Functional Requirements

The Smart Inventory Management System shall provide the following functionalities:

ID	Functional Requirement
FR-01	The system shall allow authorized users to log in securely.
FR-02	The system shall allow users to add new inventory products.
FR-03	The system shall allow updating existing product information.
FR-04	The system shall allow deleting products from inventory.
FR-05	The system shall allow searching products by name or identifier.
FR-06	The system shall display real-time inventory information.
FR-07	The system shall generate inventory reports for administrators.
FR-08	The Alert Service shall notify users when stock reaches a predefined threshold.
FR-09	The Monitoring System shall collect metrics from all microservices using Prometheus.
FR-10	Grafana shall visualize system metrics and application performance through dashboards.
FR-11	Jenkins shall automatically build, test, and deploy the application after code changes.
FR-12	Kubernetes shall manage and orchestrate all application containers.
FR-13	NGINX shall route incoming requests to the appropriate backend services.
FR-14	PostgreSQL shall securely store inventory data.
FR-15	The system shall expose RESTful APIs for communication between microservices.

4.3.2 Non-Functional Requirements

Performance

- The average API response time should be less than **200 ms**.
- The system should support multiple concurrent users.
- Monitoring metrics should refresh in near real-time.

Availability

- System uptime should be greater than **99.9%**.
- Kubernetes should automatically restart failed containers.

- Monitoring services should remain continuously available.

Scalability

- Microservices should support horizontal scaling using Kubernetes.
- New replicas can be added without modifying the application code.
- Infrastructure should support future expansion.

Security

- Only authenticated users can access protected resources.
- Communication between services uses secure HTTP endpoints.
- Sensitive configuration values are stored using Kubernetes Secrets and ConfigMaps.

Reliability

- Docker containers provide isolated execution environments.
- Prometheus continuously monitors service health.
- Grafana dashboards provide real-time system visibility.
- Alertmanager notifies administrators when failures occur.

Maintainability

- The project follows the Microservices Architecture.
- Source code is maintained using Git.
- CI/CD pipelines automate build and deployment processes.
- Infrastructure is managed as code using Terraform and Ansible.

Usability

- The web interface is simple and user-friendly.
- Dashboards display real-time metrics in an organized format.
- Navigation between system pages is straightforward.

Compatibility

The system is compatible with:

- Docker
- Kubernetes
- Linux (Ubuntu)
- PostgreSQL
- Modern web browsers (Chrome, Edge, Firefox)
- Microsoft Azure Cloud

4.4 Software Architecture

4.4.1 Architecture Overview

The Smart Inventory Management System is designed using a **Microservices Architecture** to provide scalability, flexibility, and maintainability. Each service is deployed independently inside Docker containers and orchestrated using Kubernetes. The application is hosted on Microsoft Azure, while Jenkins automates the CI/CD pipeline. Infrastructure provisioning is managed using Terraform and Ansible, and Prometheus with Grafana provides real-time monitoring and visualization.

The architecture separates the application into multiple layers, allowing each component to communicate through REST APIs while remaining independently deployable and scalable.

4.4.2 Microservices Architecture

The system follows the Microservices Architecture pattern, where every major business function is implemented as an independent service.

The application consists of the following services:

- Backend Service
- Inventory Service
- Alert Service
- PostgreSQL Database
- Prometheus Monitoring
- Grafana Dashboard
- NGINX Reverse Proxy
- Jenkins CI/CD Pipeline

Each microservice can be updated, deployed, monitored, and scaled independently without affecting other services.

4.4.3 Frontend

The frontend is developed using **React.js** and provides a responsive web interface for inventory management.

Main responsibilities include:

- User authentication
- Inventory management
- Product search
- Dashboard visualization
- REST API communication with backend services

4.4.4 Backend

The backend consists of multiple ASP.NET Core microservices.

Main services include:

- Backend API
- Inventory API
- Alert API

Responsibilities include:

- Business logic processing
- Inventory operations
- Alert generation
- REST API endpoints
- Database communication

All backend services are containerized using Docker.

4.4.5 Database

The application uses **PostgreSQL** as the primary relational database.

The database stores:

- Product information
- Inventory quantities
- User data
- Alert records
- Transaction history

Persistent storage is provided through Kubernetes Persistent Volumes.

4.4.6 Monitoring

Application monitoring is implemented using:

- Prometheus
- Grafana
- Alertmanager

Prometheus continuously collects metrics from all microservices.

Grafana visualizes:

- Request rate
- Response time
- Error rate
- Active requests
- Service health
- HTTP status codes

Alertmanager generates notifications whenever abnormal system behavior is detected.

4.4.7 CI/CD

Continuous Integration and Continuous Deployment are implemented using **Jenkins**.

The deployment workflow is:

1. Developer pushes code to GitHub.
2. Jenkins automatically starts the pipeline.
3. Application is built.
4. Docker images are created.
5. Images are pushed to the container registry.

- 6. Kubernetes updates the deployment.
- 7. Prometheus starts monitoring the new version automatically.

This process minimizes manual deployment and ensures continuous delivery.

4.4.8 Kubernetes Architecture

Kubernetes manages the entire application deployment.

Main Kubernetes resources include:

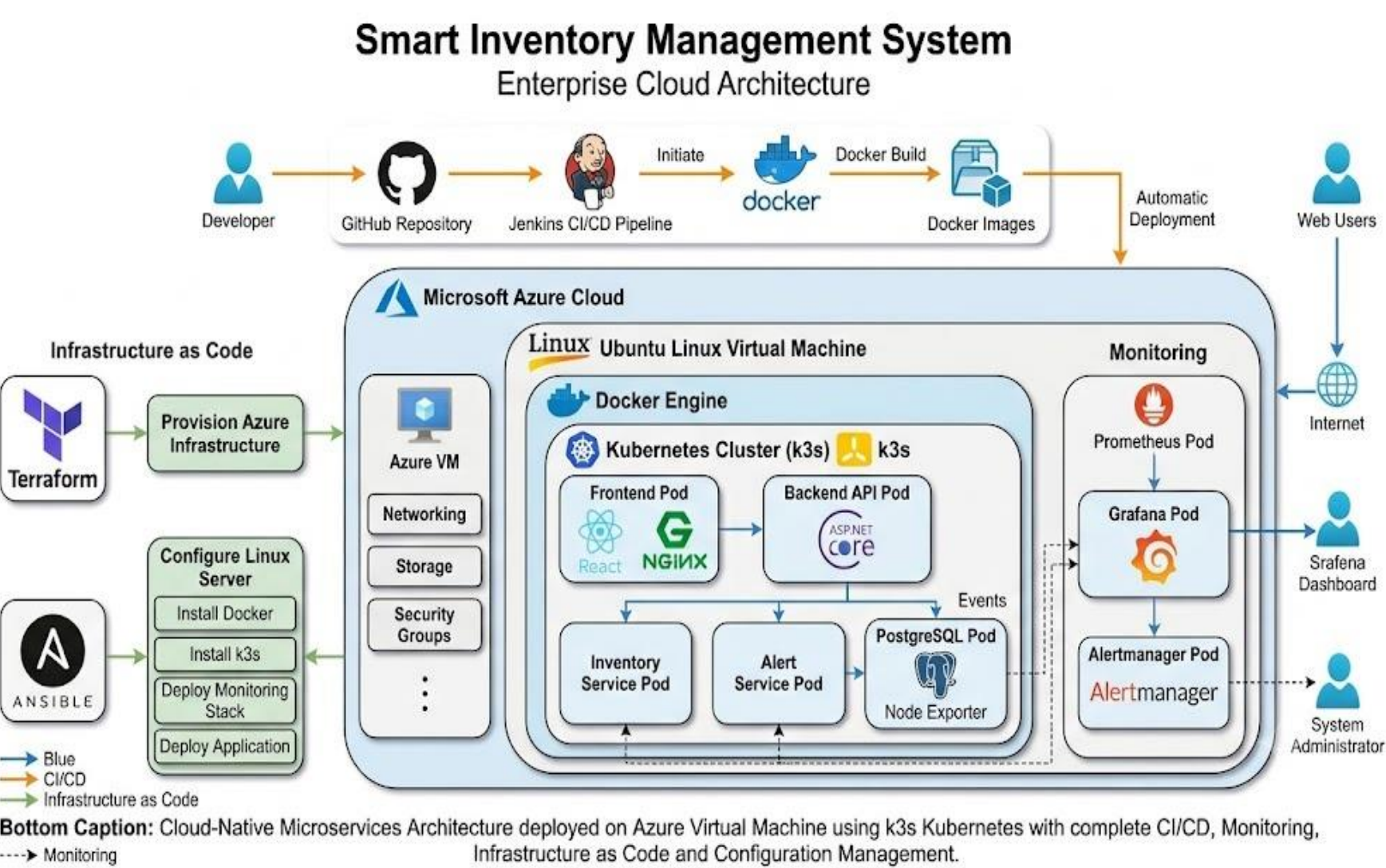
- Deployments
- Pods
- Services
- ConfigMaps
- Secrets
- Persistent Volumes
- Ingress Controller (NGINX)

Kubernetes provides:

- Automatic container restart
- Load balancing
- Service discovery
- Horizontal scaling
- Rolling updates
- High availability

4.4.9 Architecture Diagram

Figure 4.1 illustrates the overall architecture of the Smart Inventory Management System.



4.5 Database Design & Data Modeling

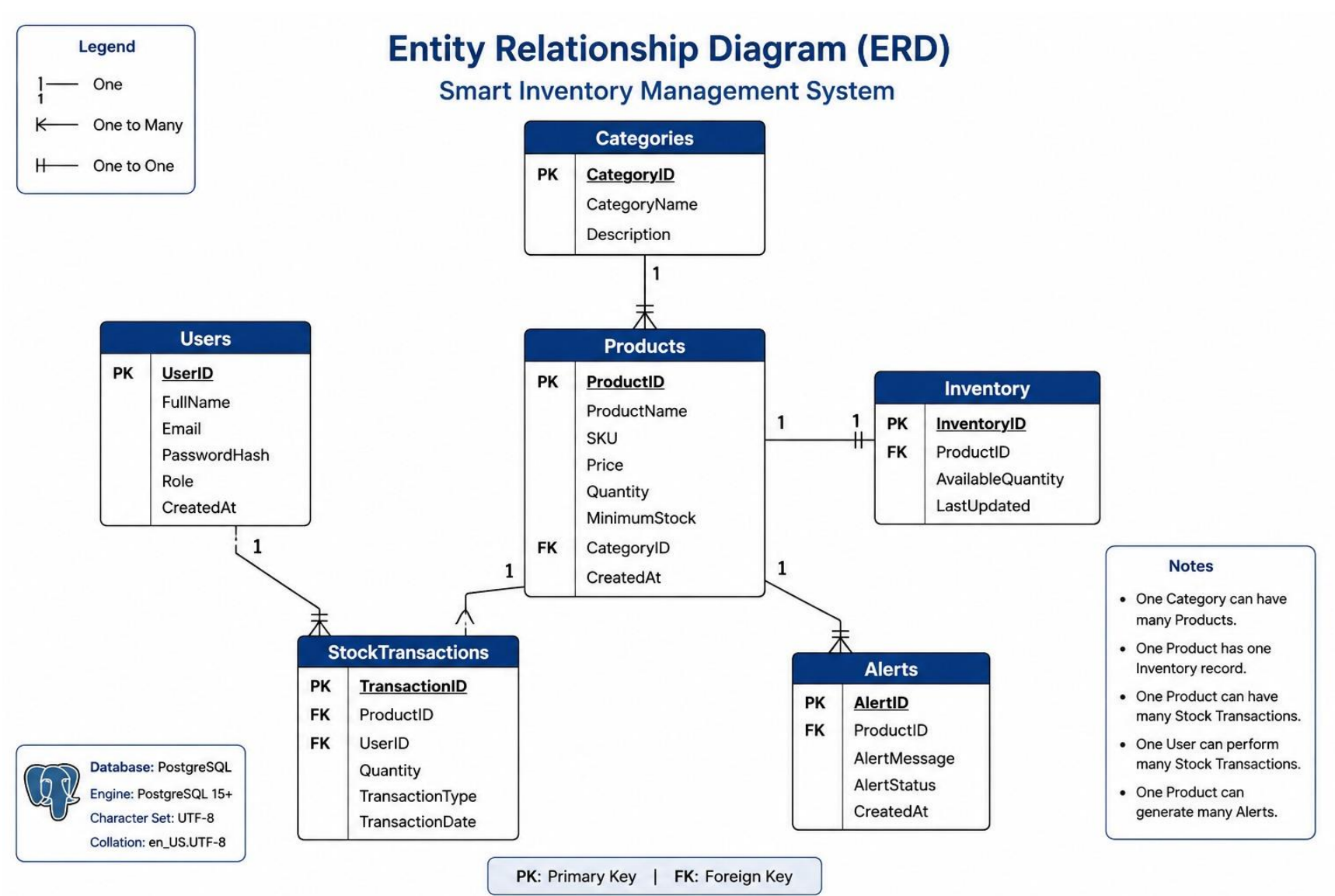
4.5.1 Database Overview

The Smart Inventory Management System uses **PostgreSQL** as the primary relational database management system. PostgreSQL was selected because of its high performance, reliability, ACID compliance, and strong support for relational data modeling.

The database stores all business data related to inventory items, stock movements, users, alerts, and transactions while ensuring data consistency and integrity across all system components.

4.5.2 ER Diagram

Figure 4.X: Entity Relationship Diagram (ERD)



4.5.3 Tables

The database consists of the following main tables:

Table	Description
Users	Stores user accounts and authentication information.
Products	Stores inventory product information.
Categories	Stores product categories.
Inventory	Stores current stock quantities.
StockTransactions	Records stock in/out operations.
Alerts	Stores low-stock alerts generated by the Alert Service.

4.5.4 Attributes

Users

- UserID
- FullName
- Email
- PasswordHash
- Role
- CreatedAt

Categories

- CategoryID
- CategoryName
- Description

Products

- ProductID
- ProductName
- SKU
- Price
- Quantity
- MinimumStock
- CategoryID
- CreatedAt

Inventory

- InventoryID
- ProductID
- AvailableQuantity
- LastUpdated

StockTransactions

- TransactionID
- ProductID
- Quantity
- TransactionType
- TransactionDate
- UserID

Alerts

- AlertID
- ProductID
- AlertMessage
- AlertStatus
- CreatedAt

4.5.5 Relationships

The database entities are related as follows:

- One Category can contain many Products.
- One Product has one Inventory record.
- One Product can have many Stock Transactions.
- One Product can generate many Alerts.
- One User can perform many Stock Transactions.

4.5.6 Primary Keys

Table	Primary Key
Users	UserID
Categories	CategoryID
Products	ProductID
Inventory	InventoryID
StockTransactions	TransactionID
Alerts	AlertID

4.5.7 Foreign Keys

Table	Foreign Key	References
Products	CategoryID	Categories(CategoryID)
Inventory	ProductID	Products(ProductID)
StockTransactions	ProductID	Products(ProductID)
StockTransactions	UserID	Users(UserID)
Alerts	ProductID	Products(ProductID)

4.5.8 Normalization

The database is designed according to the **Third Normal Form (3NF)** to reduce redundancy and improve data consistency.

The normalization process ensures that:

- Each table represents a single entity.
- Every non-key attribute depends only on the primary key.
- Duplicate data is minimized.
- Data integrity is maintained through foreign key constraints.
- The database supports efficient querying and future scalability.

4.6 Data Flow & System Behavior

4.6.1 Context DFD

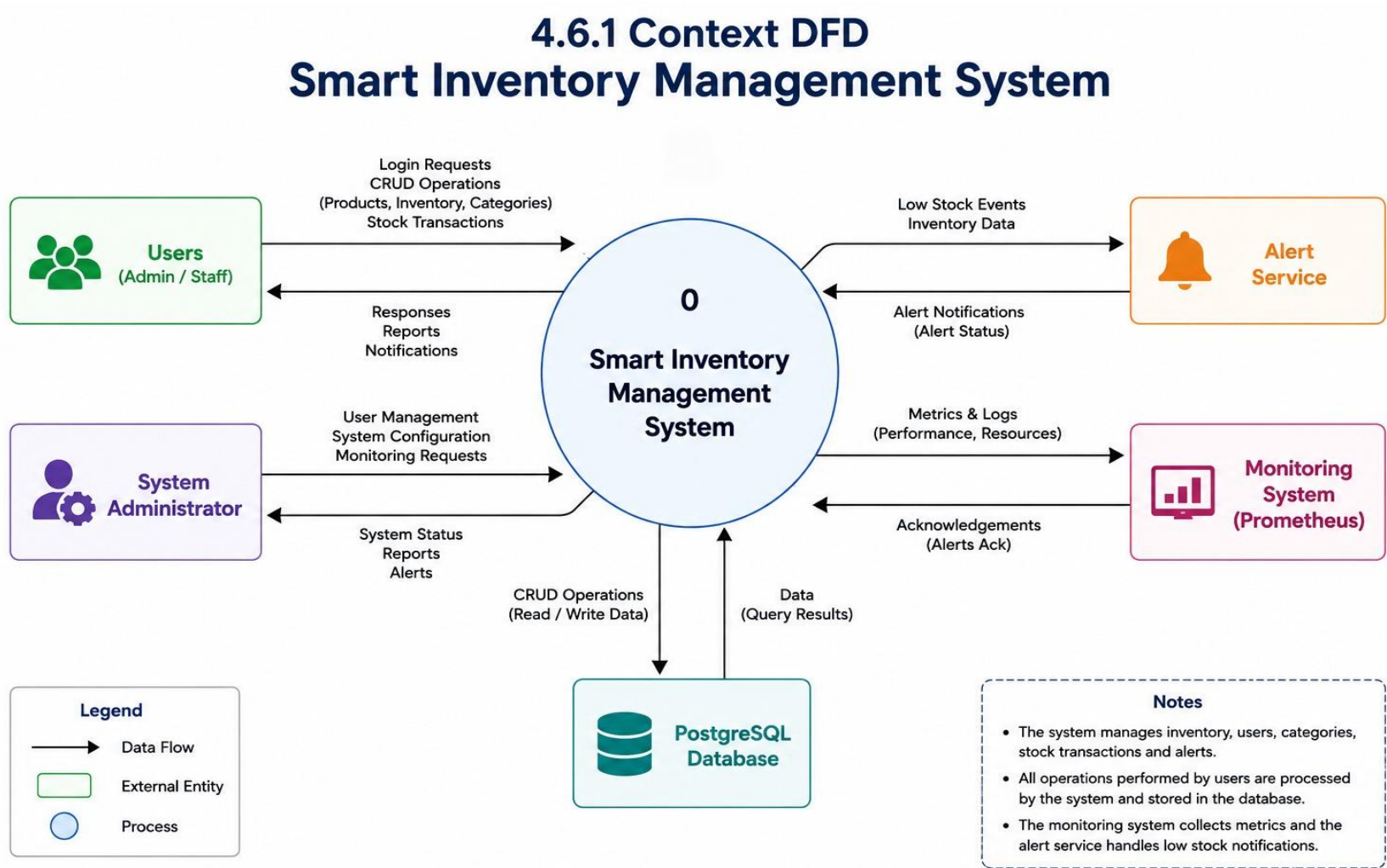


Figure 4.6.1: Context Data Flow Diagram (DFD)

4.6.2 Level-1 DFD

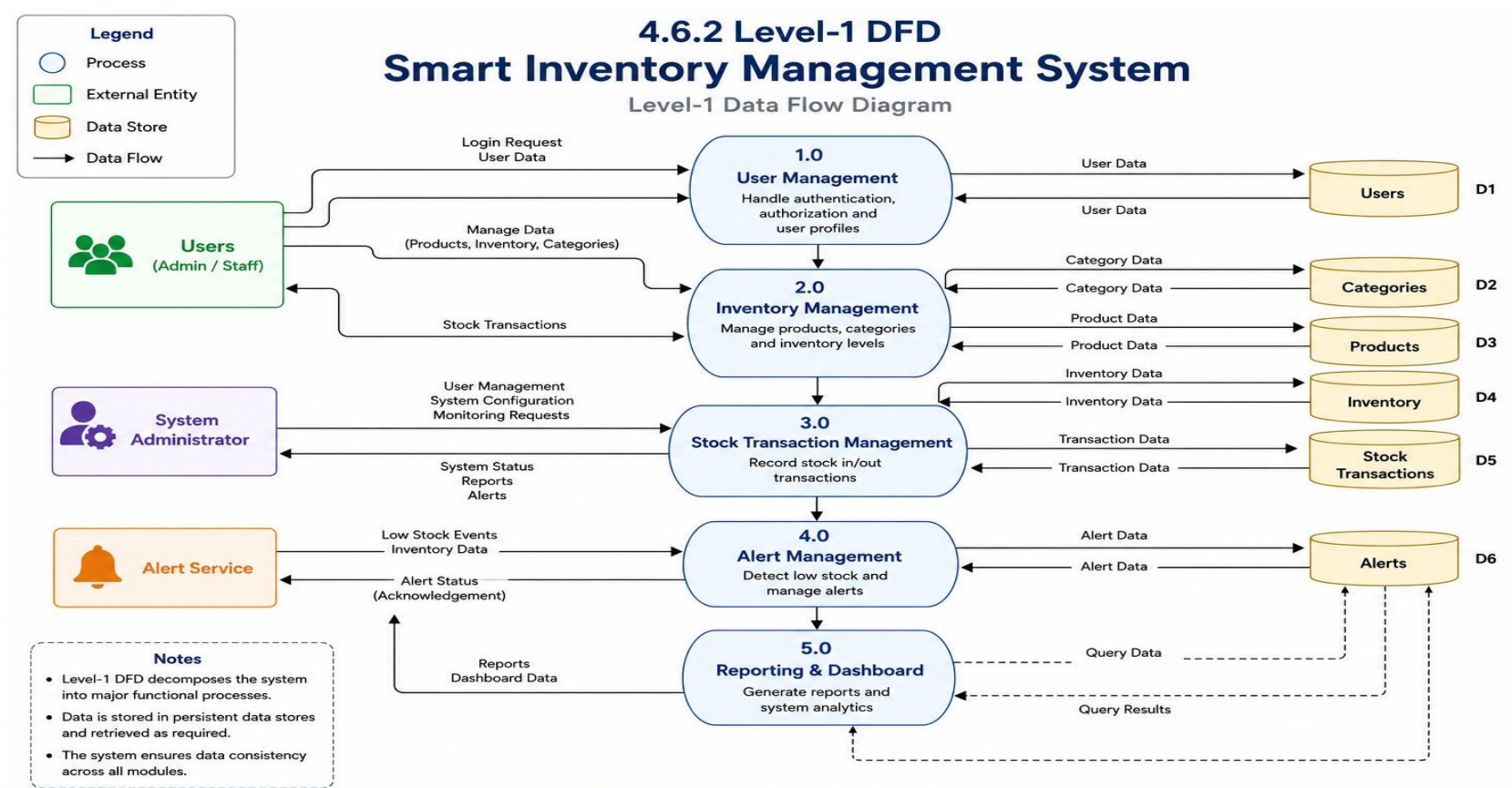
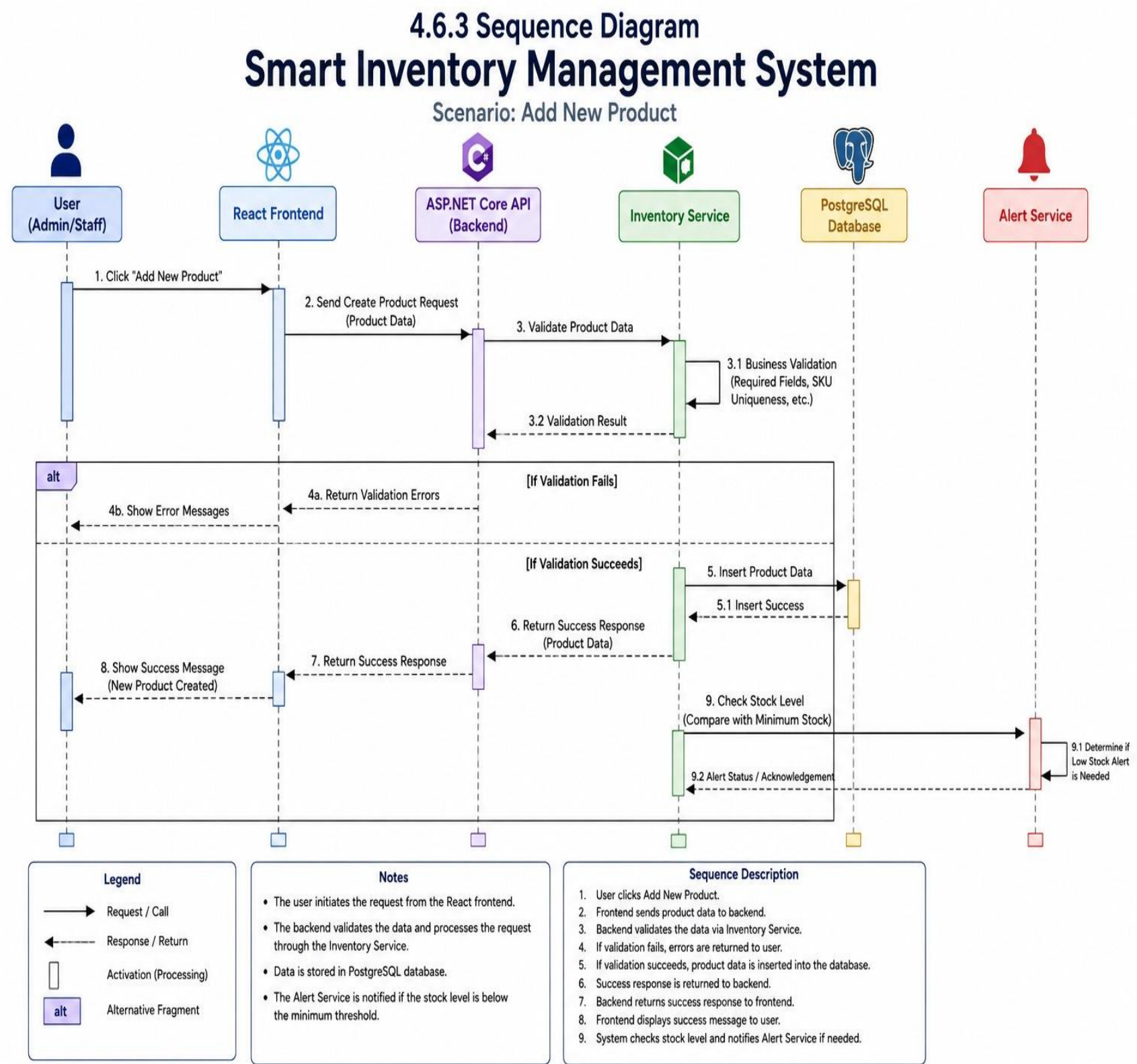


Figure 4.6.2: Level-1 Data Flow Diagram (DFD)

4.6.3 Sequence Diagram



4.6.4 Activity Diagram

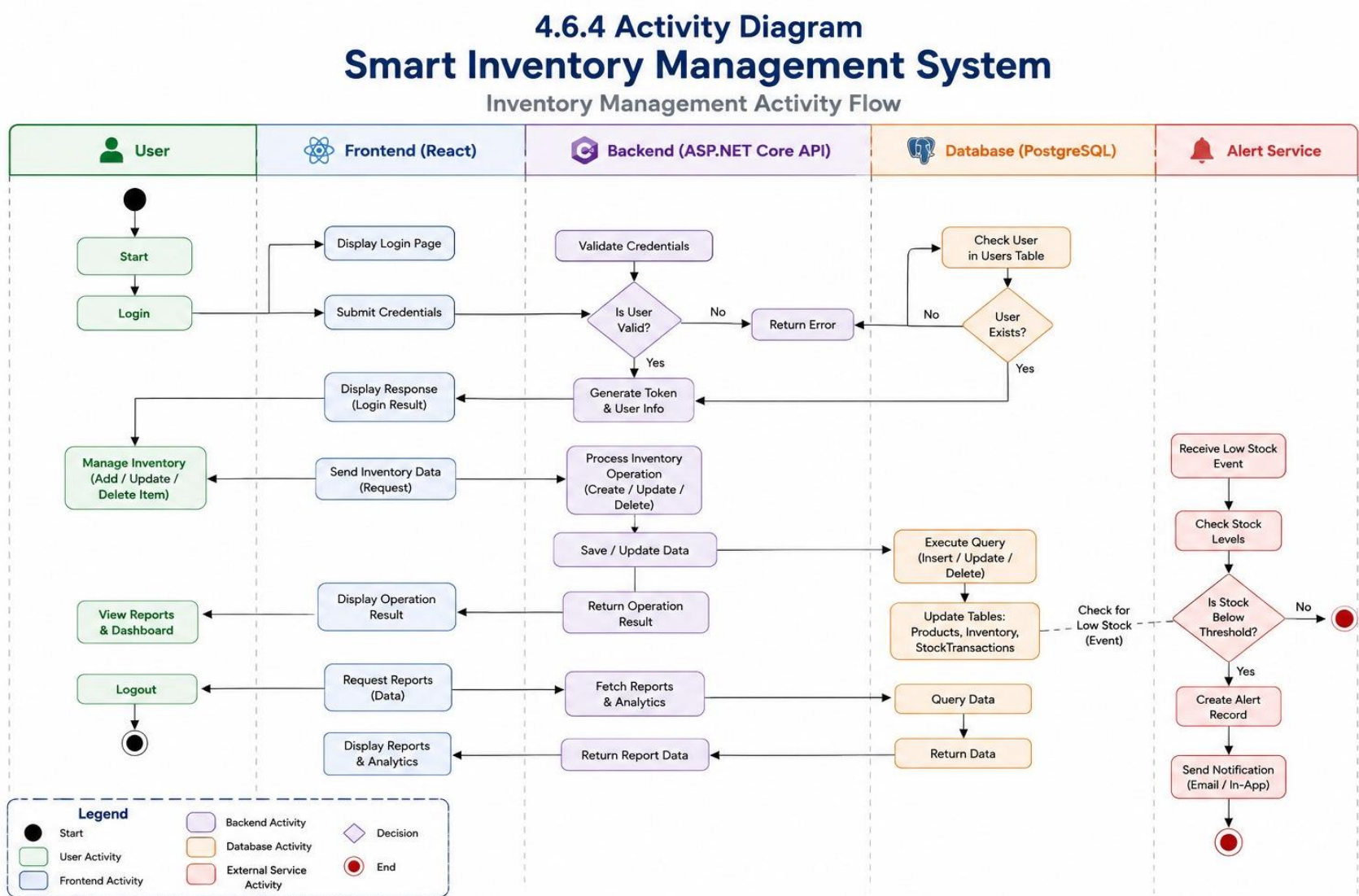
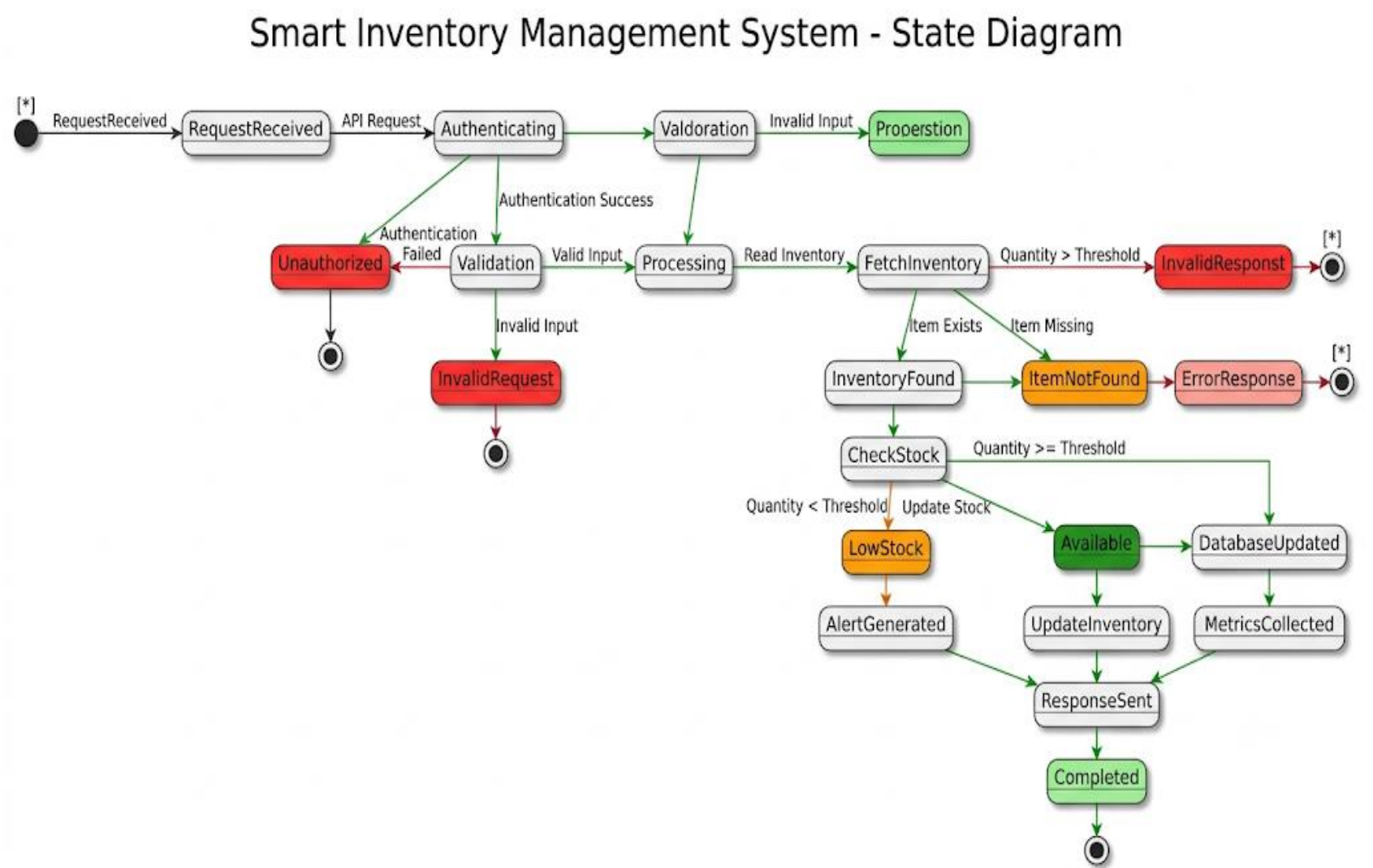
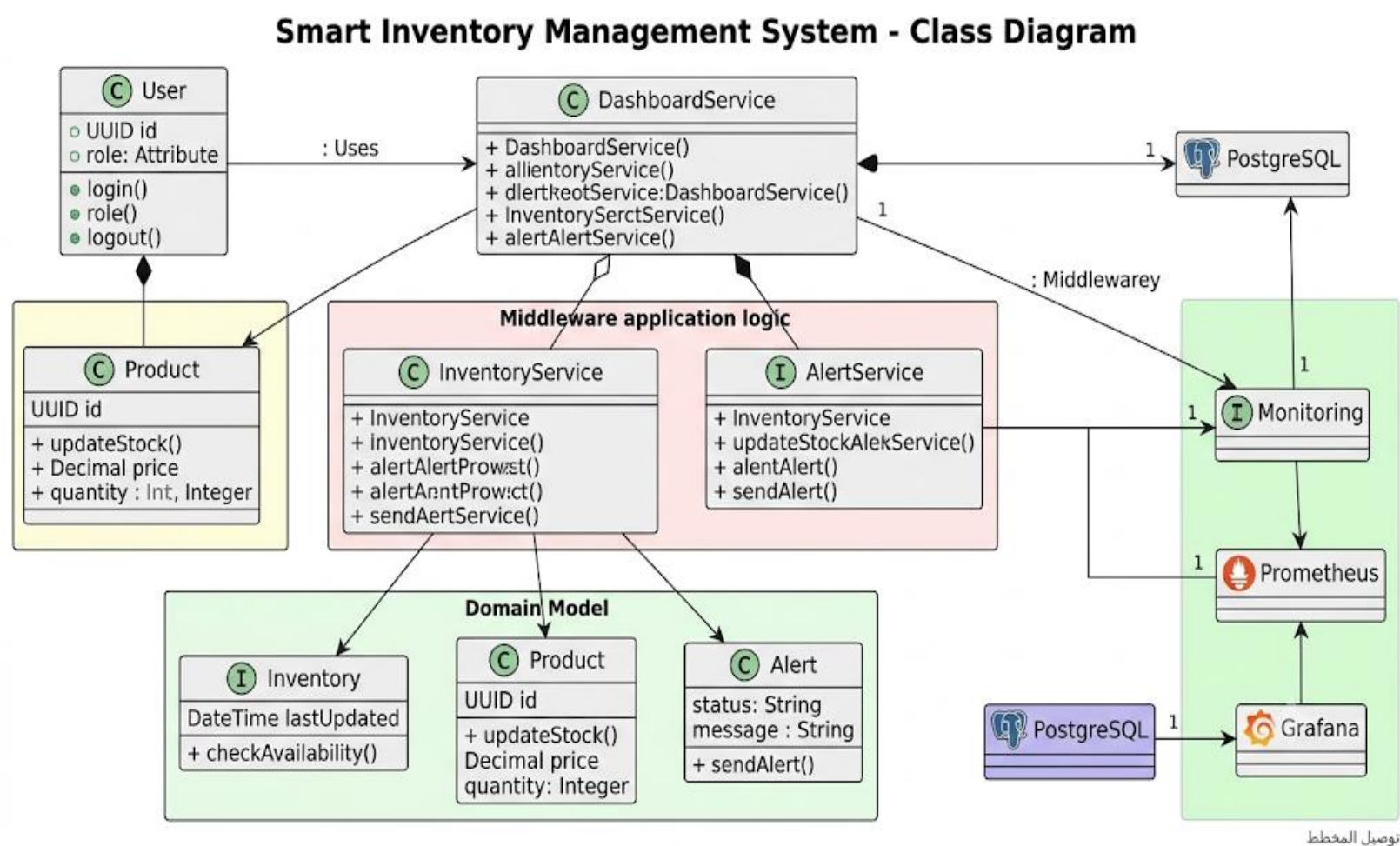


Figure 4.6.4: Activity Diagram – Inventory Management Process

4.6.5 State Diagram



4.6.6 Class Diagram



4.7 UI/UX Design

4.7.1 Design Principles

The Smart Inventory Management System was designed with a focus on simplicity, usability, and efficiency. The user interface provides quick access to all system functions while maintaining a clean and modern appearance.

The main design principles include:

- Clean and minimal interface.
- Consistent navigation across all pages.
- Fast access to frequently used features.
- Clear visual hierarchy.
- Easy-to-read typography.
- Responsive layout for different screen sizes.
- Dark theme to reduce eye strain during long working sessions.

4.7.2 Color Palette

The application uses a modern dark color scheme to improve readability and provide a professional appearance.

Element	Color
Background	Dark Navy (#0B1023)
Sidebar	Deep Blue (#121B3B)
Primary Color	Purple (#6C63FF)
Success	Green (#22C55E)
Warning	Orange (#F59E0B)
Error	Red (#EF4444)
Text	White / Light Gray

4.7.3 Typography

The interface uses a clean sans-serif font to maximize readability.

Typography hierarchy:

- Dashboard Titles → Large Bold
- Section Titles → Medium Bold
- Table Headers → Medium
- Body Text → Regular
- Buttons → Semi-Bold

This typography improves readability while maintaining consistency throughout the application.

4.7.4 Responsive Design

The frontend was developed using responsive design principles to ensure compatibility with different devices and screen resolutions.

Responsive features include:

- Flexible dashboard layout.
- Responsive navigation sidebar.
- Adaptive tables.
- Responsive charts.
- Mobile-friendly cards.
- Automatic component resizing.

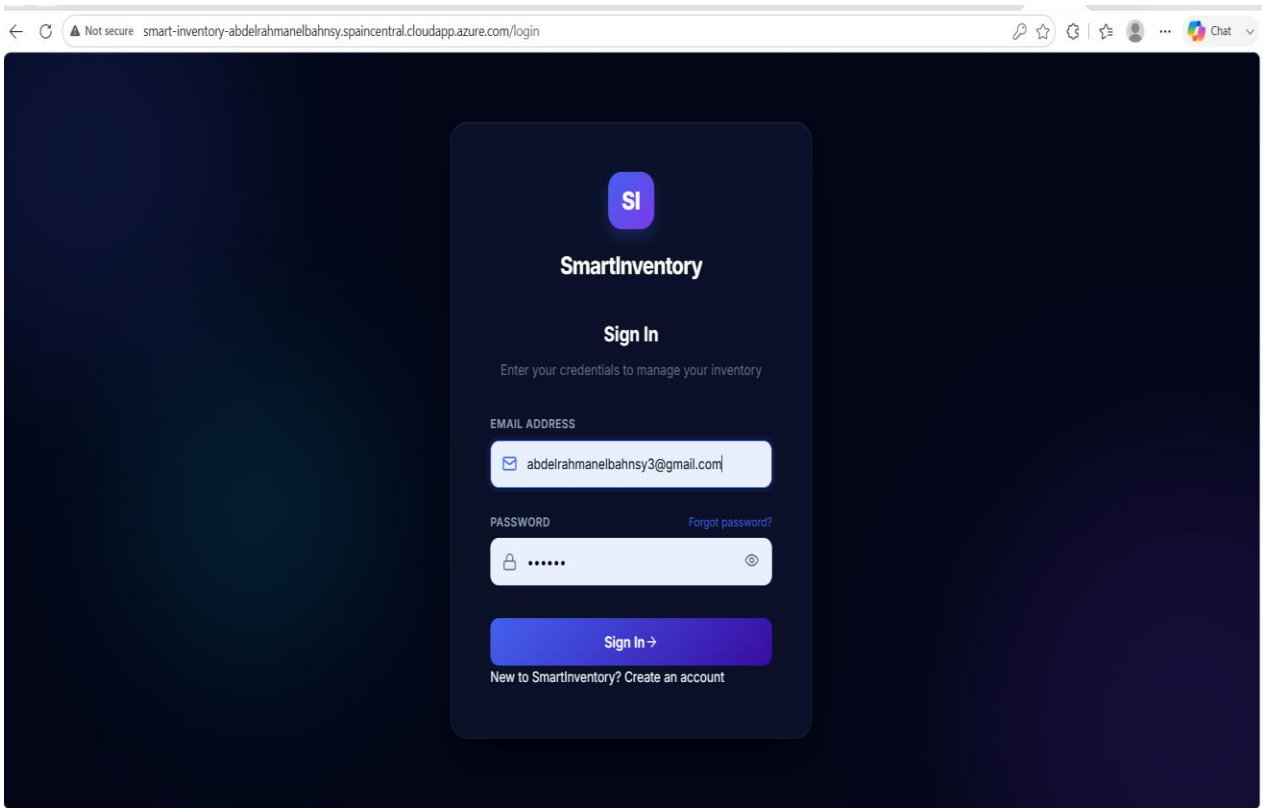
4.7.5 Wireframes

During the design phase, wireframes were created to define the layout and user flow before implementation.

The wireframes included:

- Login Screen
- Dashboard
- Inventory Management
- Products Page
- Orders Page
- Alerts Page

- Reports Page

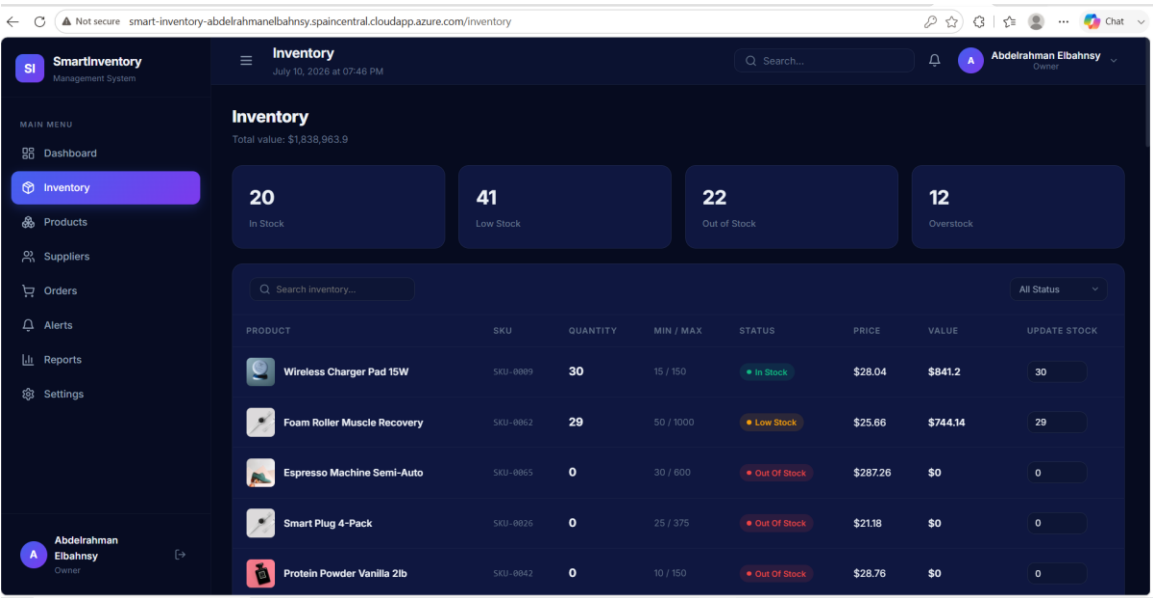


4.7.6 Mockups

After completing the wireframes, high-fidelity mockups were prepared to visualize the final appearance of the system before development.

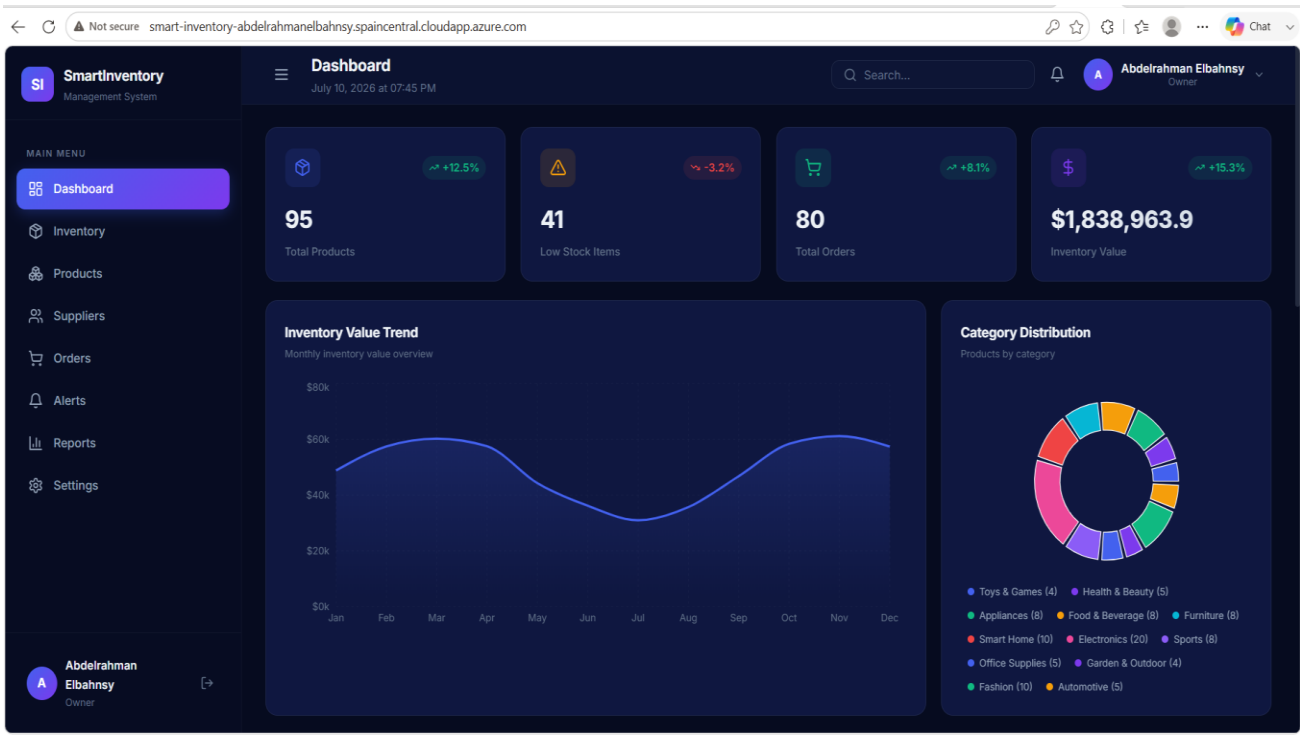
The mockups included:

- Modern Dashboard
- Product Management
- Inventory Overview
- Order Management
- Alert Monitoring



4.7.7 Screenshots (Real System)

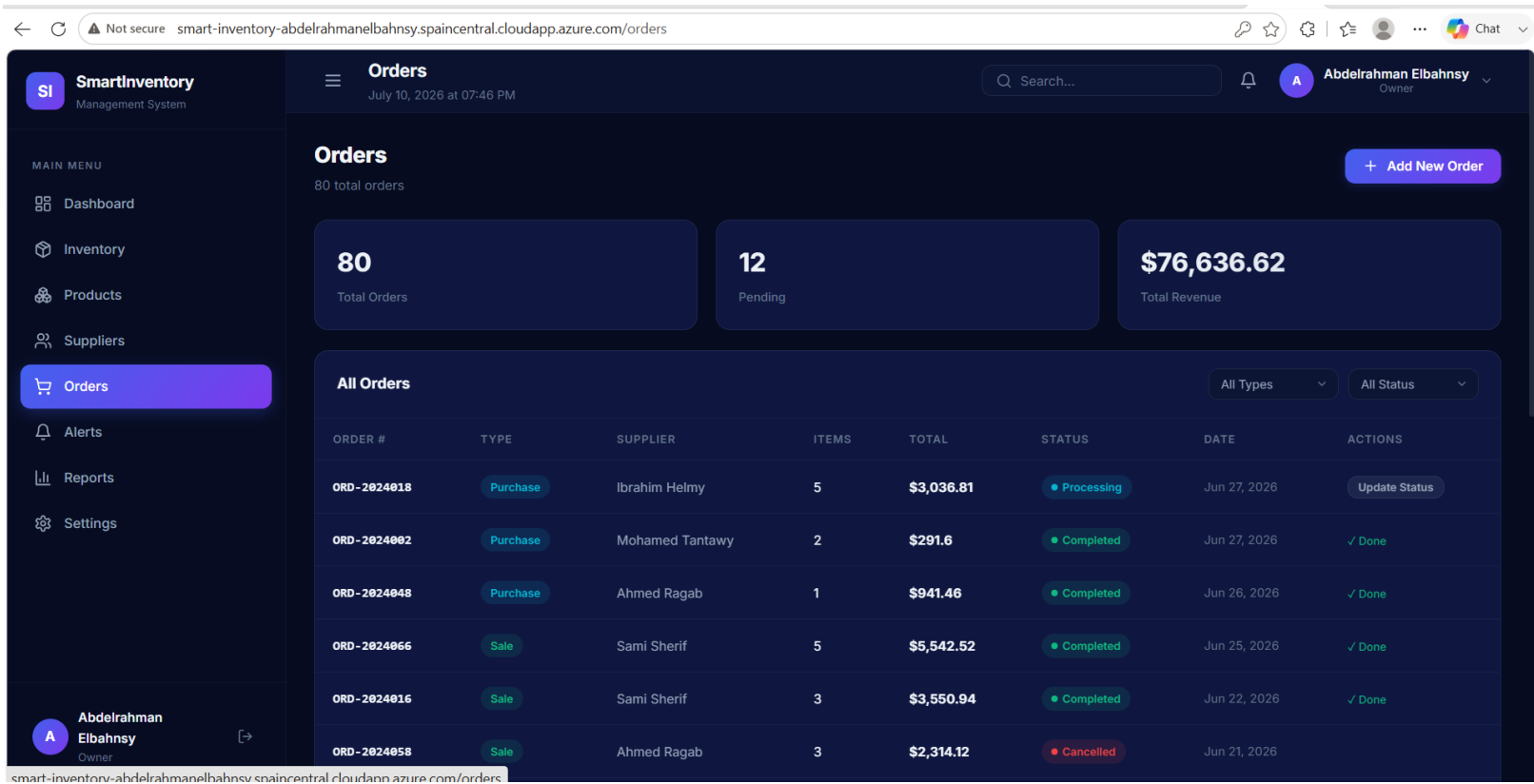
Figure 4.17 Dashboard



Caption

Figure 4.17: Smart Inventory Dashboard showing inventory statistics, inventory value trend, category distribution, and key performance indicators.

Figure 4.18 Orders Module



Caption

Figure 4.18: Orders Management Module displaying order information, suppliers, order status, revenue statistics, and filtering options.

4.8 System Deployment & Integration

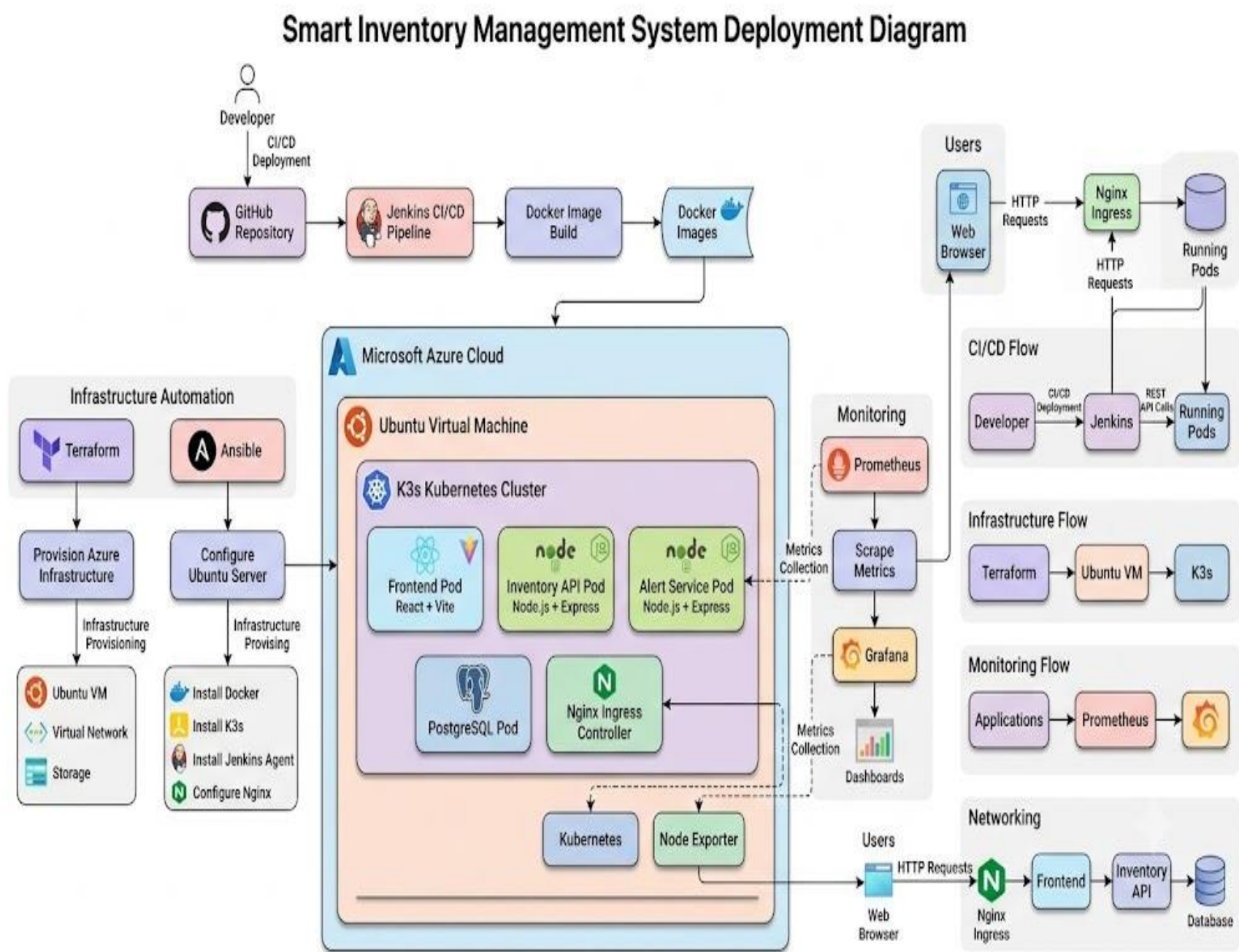
4.8.1 Technology Stack

The Smart Inventory Management System was built using a modern cloud-native technology stack that supports scalability, automation, monitoring, and continuous deployment.

Layer	Technology
Frontend	React.js, Vite, Tailwind CSS
Backend	Node.js, Express.js
Database	PostgreSQL
Containerization	Docker
Container Orchestration	Kubernetes (K3s)
Reverse Proxy	Nginx
CI/CD	Jenkins
Infrastructure as Code	Terraform
Configuration Management	Ansible
Monitoring	Prometheus
Visualization	Grafana
Source Control	Git & GitHub
Cloud Platform	Microsoft Azure (Ubuntu VM)

4.8.2 Deployment Diagram

The application is deployed on a Microsoft Azure Virtual Machine running Ubuntu Linux. All application services are containerized using Docker and orchestrated using a K3s Kubernetes cluster. Jenkins automates the CI/CD pipeline, while Terraform provisions the infrastructure and Ansible performs server configuration. Prometheus continuously collects application metrics, and Grafana visualizes the monitoring dashboards. **Figure 4.**



4.8.3 Component Diagram

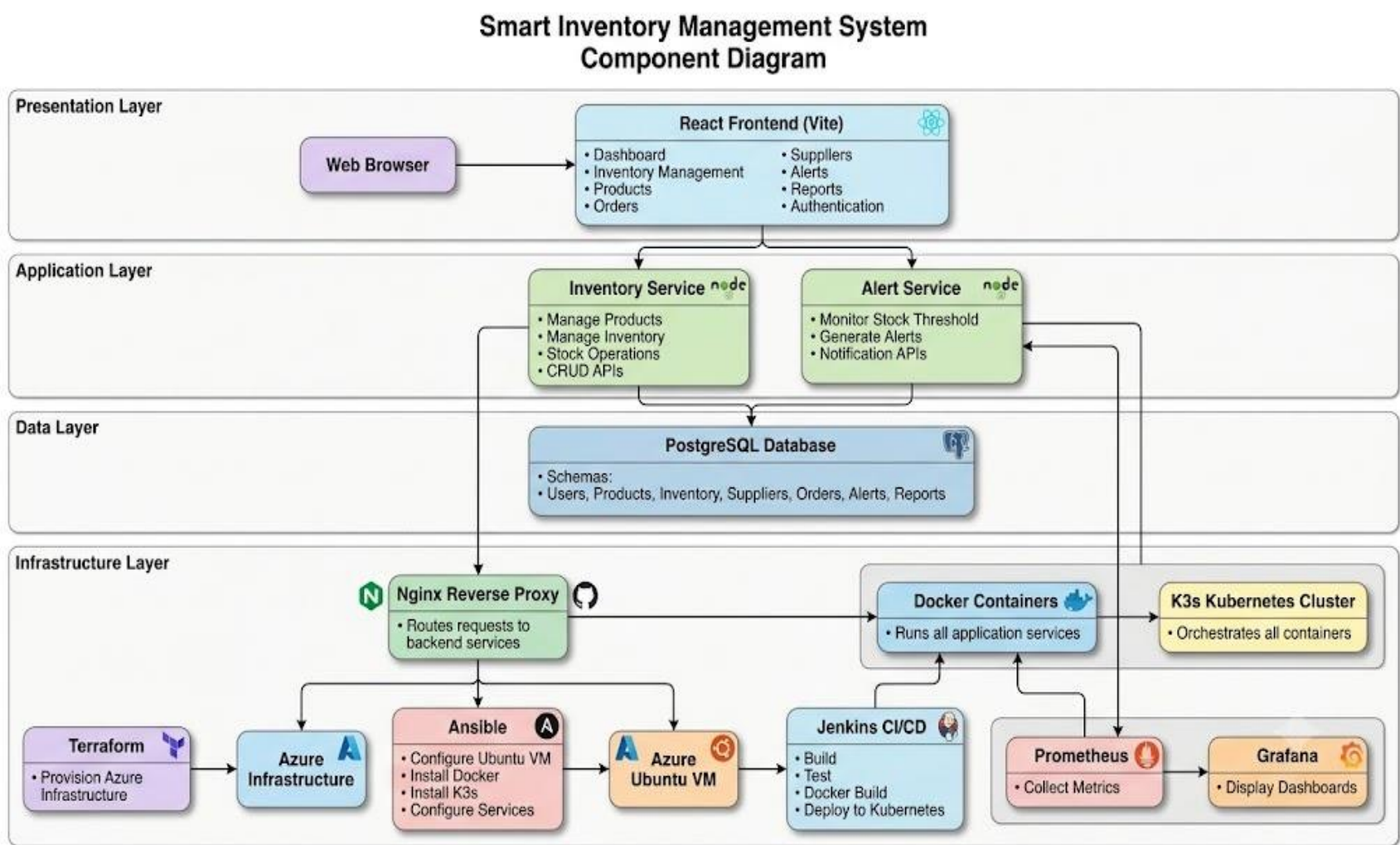
The system follows a Microservices Architecture where each component is responsible for a specific business function.

Main Components:

- React Frontend
- Inventory API Service
- Alert API Service
- PostgreSQL Database
- Nginx Reverse Proxy
- Prometheus Monitoring
- Grafana Dashboard
- Jenkins CI/CD Pipeline

The components communicate through REST APIs and are deployed independently inside Kubernetes pods.

Figure 4



4.8.4 Infrastructure Overview

The infrastructure was designed according to cloud-native best practices to ensure reliability, scalability, and maintainability.

The deployment workflow is summarized as follows:

1. Developers push source code to GitHub.
2. Jenkins automatically detects repository changes.
3. Jenkins builds Docker images for the services.
4. Docker images are deployed to the K3s Kubernetes cluster.
5. Nginx routes incoming requests to the appropriate microservice.
6. PostgreSQL stores application data.
7. Prometheus collects metrics from all services.

- 8. Grafana visualizes real-time monitoring dashboards.
- 9. Terraform provisions Azure infrastructure.
- 10. Ansible configures the virtual machine and required services.

Infrastructure Features

- Cloud deployment on Microsoft Azure.
- Kubernetes (K3s) orchestration.
- Dockerized microservices.
- Automated CI/CD pipeline.
- Infrastructure as Code using Terraform.
- Automated server configuration using Ansible.
- Real-time monitoring with Prometheus and Grafana.
- Secure HTTP routing through Nginx.
- Centralized PostgreSQL database.
- GitHub-based version control.

4.9 API Documentation

The Smart Inventory Management System exposes a set of RESTful APIs that allow communication between the frontend, backend microservices, and monitoring components. All APIs exchange data using JSON over HTTP.

4.9.1 Authentication API

The Authentication API is responsible for user authentication and authorization.

Method	Endpoint	Description
POST	/api/auth/login	User Login
POST	/api/auth/register	Create New User
GET	/api/auth/profile	Get User Profile
POST	/api/auth/logout	Logout User

Example Request

POST /api/auth/login

Content-Type: application/json

```
{
  "email": "admin@example.com",
  "password": "123456"
}
```

Example Response

```
{
  "success": true,
  "token": "JWT_TOKEN",
  "user": {
    "id": 1,
    "name": "Abdelrahman",
    "role": "Admin"
  }
}
```

4.9.2 Inventory API

The Inventory API manages products and inventory operations.

Method	Endpoint	Description
GET	/api/inventory	Get All Products
GET	/api/inventory/{id}	Get Product
POST	/api/inventory	Add Product
PUT	/api/inventory/{id}	Update Product
DELETE	/api/inventory/{id}	Delete Product

Example Request

```
POST /api/inventory
{
  "name": "Laptop",
  "quantity": 20,
  "price": 1200,
  "supplier": "Dell"
}
```

Example Response

```
{
  "success": true,
  "message": "Product Created Successfully"
}
```

4.9.3 Alert API

The Alert Service generates and retrieves inventory alerts

Method	Endpoint	Description
GET	/api/alerts	Get Alerts
POST	/api/alerts/check	Check Stock Threshold
PUT	/api/alerts/{id}	Update Alert Status

Example Request

```
POST /api/alerts/check
{
  "productId":15
}
```

Example Response

```
{
  "status":"Low Stock",
  "message":"Product quantity is below threshold."
}
```

4.9.4 Metrics API

The Metrics API exposes Prometheus metrics for monitoring purposes.

Method	Endpoint	Description
GET	/metrics	Export Prometheus Metrics

Example:

```
GET /metrics
```

Example Response

```
http_requests_total 1520

http_request_duration_seconds 0.042

active_requests 5

inventory_items_total 95

low_stock_products 41
```

4.9.5 Request & Response Examples

Create Product

Request

```
POST /api/inventory
{
  "name": "Keyboard",
  "price": 75,
  "quantity": 15,
  "supplier": "Logitech"
}
```

Response

```
{
  "success": true,
  "productId": 101,
  "message": "Product added successfully."
}
```

Update Inventory

Request

```
PUT /api/inventory/101
{
  "quantity": 12
}
```

Response

```
{
  "success": true,
  "message": "Inventory updated successfully."
}
```

Get Alerts

Request

```
GET /api/alerts
```

Response

```
[
  {
    "id": 1,
    "product": "Laptop",
    "quantity": 3,
    "status": "Low Stock"
  }
]
```

Retrieve Metrics

Request

```
GET /metrics
```

Response

```
http_requests_total 2145
```

```
active_requests 4
```

```
http_request_duration_seconds 0.038
```

```
inventory_items_total 95
```

```
low_stock_products 41
```

4.10 Testing & Validation

To ensure the reliability, stability, and correctness of the Smart Inventory Management System, different testing techniques were applied throughout the development lifecycle. These tests verify that individual components, integrated services, and the complete system function correctly under normal operating conditions.

4.10.1 Unit Testing

Unit testing was performed to verify the functionality of individual modules independently. Each API endpoint and business logic function was tested to ensure it produced the expected output for valid and invalid inputs.

The following components were tested:

- Authentication Module
- Inventory Service
- Alert Service
- Product Management Functions
- Database Operations

Objectives

- Verify individual functions.
- Detect coding errors early.
- Validate API responses.
- Ensure business logic correctness.

4.10.2 Integration Testing

Integration testing was conducted to validate communication between the system components after combining the individual modules.

The following integrations were verified:

- React Frontend ↔ Inventory API

- React Frontend ↔ Alert API
- Inventory Service ↔ PostgreSQL
- Alert Service ↔ PostgreSQL
- Prometheus ↔ Application Services
- Grafana ↔ Prometheus
- Jenkins ↔ Kubernetes Deployment

Objectives

- Verify API communication.
- Validate database connectivity.
- Ensure successful microservice interaction.
- Test monitoring integration.

4.10.3 System Testing

System testing was performed after deploying the complete application on the K3s Kubernetes cluster hosted on Microsoft Azure.

The following aspects were evaluated:

- User Authentication
- Inventory Management
- Order Processing
- Alert Generation
- Dashboard Functionality
- Monitoring Services
- CI/CD Deployment
- Kubernetes Deployment
- Database Connectivity

The complete system operated successfully with all services communicating correctly through REST APIs.

4.10.4 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) was performed to ensure that the implemented system satisfies the project requirements and provides a user-friendly experience.

The system was evaluated based on the following criteria:

- Ease of use
- User interface responsiveness
- Inventory management workflow
- Dashboard usability
- Alert notification accuracy
- Overall system performance

The testing results confirmed that the application met the functional and non-functional requirements defined during the analysis phase.

Table 4.X Testing Summary

Test Type	Objective	Result
-----------	-----------	--------

Test Type	Objective	Result
Unit Testing	Verify individual functions	Passed
Integration Testing	Validate communication between services	Passed
System Testing	Verify complete system functionality	Passed
User Acceptance Testing	Ensure user requirements are satisfied	Passed

Validation Summary

The Smart Inventory Management System was successfully validated after deployment on the Azure virtual machine using Docker and K3s Kubernetes. All core functionalities, including inventory management, alert generation, monitoring, and automated deployment, operated as expected. The testing process demonstrated that the system is stable, reliable, and suitable for production-like environments.

4.11 Deployment Strategy

The Smart Inventory Management System follows a cloud-native deployment strategy based on containerization, Kubernetes orchestration, Infrastructure as Code (IaC), and automated Continuous Integration/Continuous Deployment (CI/CD). The application is deployed on a Microsoft Azure Virtual Machine running Ubuntu Linux, providing a scalable and maintainable production environment.

4.11.1 Azure Infrastructure

The application is hosted on a Microsoft Azure Virtual Machine running Ubuntu Linux. Azure provides the cloud infrastructure required to host the Kubernetes cluster and all application services.

The Azure environment includes:

- Ubuntu Linux Virtual Machine
- Virtual Network (VNet)
- Network Security Groups (NSG)
- Public IP Address
- Persistent Storage
- Secure SSH Access

This infrastructure provides high availability, secure networking, and flexible resource management.

4.11.2 Docker Containers

Each system component is packaged as an independent Docker container.

The containerized services include:

- React Frontend
- Inventory API
- Alert Service
- PostgreSQL Database

- Prometheus
- Grafana
- Nginx

Containerization ensures:

- Environment consistency
- Fast deployment
- Easy portability
- Simplified dependency management

4.11.3 Kubernetes Deployment

The application is orchestrated using a lightweight **K3s Kubernetes** cluster.

Kubernetes manages:

- Pods
- Deployments
- Services
- ConfigMaps
- Secrets
- Persistent Volumes
- Ingress Controller

Key deployment features include:

- Automatic pod recovery
- Rolling updates
- Service discovery
- Internal load balancing
- High availability
- Horizontal scalability

4.11.4 Jenkins Pipeline

A Jenkins CI/CD pipeline automates the deployment process after every code update.

Deployment workflow:

1. Developer pushes code to GitHub.
2. Jenkins detects repository changes.
3. Source code is built.
4. Docker images are generated.
5. Images are deployed to the K3s cluster.
6. Kubernetes updates running pods.
7. Prometheus immediately begins monitoring the new deployment.

This automated workflow minimizes manual intervention and accelerates software delivery.

4.11.5 Prometheus Monitoring

Prometheus continuously collects metrics from all deployed services.

The monitoring process includes:

- HTTP Request Rate
- Response Time
- Active Requests
- Error Rate
- Service Availability
- Kubernetes Metrics

Collected metrics are stored in a time-series database for analysis and visualization.

4.11.6 Grafana Dashboards

Grafana provides interactive dashboards for real-time system monitoring.

Available dashboards display:

- CPU Utilization
- Memory Usage
- Request Rate
- HTTP Response Time
- Active Requests
- Kubernetes Health
- Inventory Service Metrics
- Alert Service Metrics

These dashboards enable administrators to monitor application health and performance efficiently.

4.11.7 Nginx Reverse Proxy

Nginx acts as the entry point for incoming client requests.

Its responsibilities include:

- Reverse Proxy
- HTTP Request Routing
- Load Distribution
- Static File Serving
- Secure Communication
- Routing requests to backend microservices

Using Nginx improves system performance and simplifies service communication.

4.11.8 Scaling Strategy

The deployment architecture supports future scalability through Kubernetes.

Scaling capabilities include:

- Horizontal Pod Scaling
- Independent Microservice Scaling
- Rolling Updates
- Zero-Downtime Deployment
- Automatic Pod Recovery

- Load Distribution Across Services

This strategy enables the system to handle increasing workloads while maintaining high availability and performance.

Deployment Workflow Summary

The overall deployment process follows these steps:

1. Code is pushed to the GitHub repository.
2. Jenkins automatically starts the CI/CD pipeline.
3. Docker images are built for the updated services.
4. Kubernetes deploys the new containers to the K3s cluster.
5. Nginx routes incoming requests to the appropriate services.
6. PostgreSQL stores and retrieves application data.
7. Prometheus collects monitoring metrics.
8. Grafana visualizes system performance through real-time dashboards.

Chapter 5

Implementation (Source Code & Execution)

5.1 Source Code

This chapter describes the implementation details of the Smart Inventory Management System. The project follows a cloud-native architecture based on Microservices, where each component is implemented independently and deployed using Docker and K3s Kubernetes. The source code is organized into separate modules for the frontend, backend, database, monitoring, and infrastructure automation, ensuring maintainability, scalability, and ease of deployment.

5.1.1 Project Structure

The project follows a modular directory structure to separate application layers and infrastructure resources. Each module is responsible for a specific functionality, improving code organization and maintainability.

Main project directories include:

- **frontend/** – React application and user interface.
- **backend/** – RESTful APIs and business logic.
- **database/** – PostgreSQL schema and initialization scripts.
- **kubernetes/** – Kubernetes deployment manifests.
- **terraform/** – Infrastructure as Code configuration.
- **ansible/** – Server configuration and automation playbooks.
- **monitoring/** – Prometheus and Grafana configuration files.
- **nginx/** – Reverse proxy configuration.
- **jenkins/** – CI/CD pipeline configuration.
- **docs/** – Project documentation and diagrams.

This structure enables independent development, deployment, and maintenance of each system component.

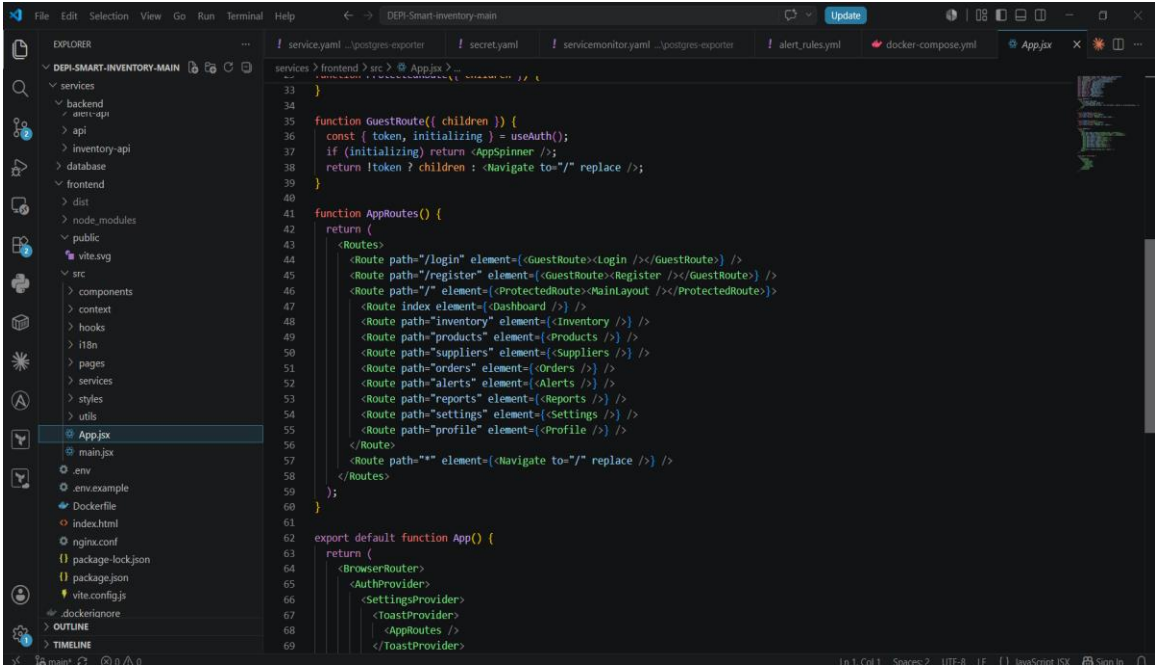
5.1.2 Frontend Implementation

The frontend was implemented using **React.js** and **Vite** to provide a fast and responsive user interface.

The frontend is responsible for:

- User authentication.
- Dashboard visualization.
- Product management.
- Inventory management.
- Order management.
- Alert display.
- Report visualization.
- Communication with backend services through REST APIs.

The application is built using reusable React components, allowing easier maintenance and future expansion.



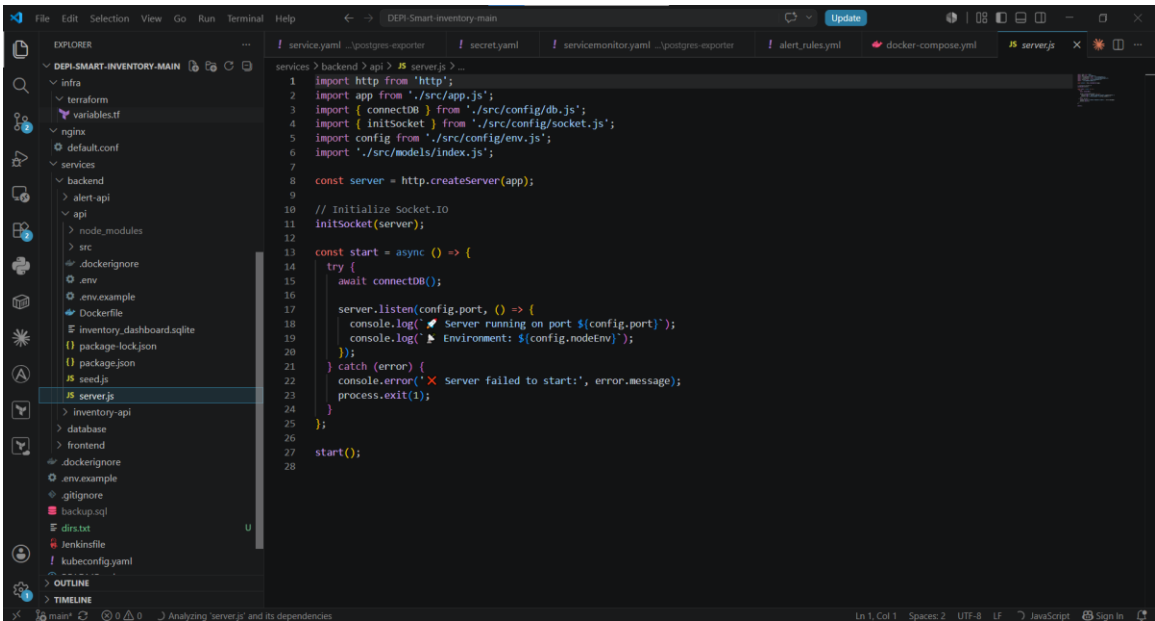
5.1.3 Backend Implementation

The backend was developed using **Node.js** and **Express.js** following the Microservices Architecture.

The backend provides RESTful APIs responsible for:

- User authentication.
- Product management.
- Inventory operations.
- Alert generation.
- Business logic processing.
- Database communication.

Each service is implemented independently, allowing isolated deployment and scaling within the Kubernetes cluster.



5.1.4 Database Implementation

The system uses **PostgreSQL** as the primary relational database.

The database stores:

- User accounts.
- Products.
- Inventory records.
- Suppliers.
- Orders.
- Alerts.

- Reports.

Relationships between entities are implemented using primary and foreign keys to ensure data consistency and integrity.

5.1.5 Microservices Implementation

The application adopts the **Microservices Architecture**, where each business function is deployed as an independent service.

Implemented services include:

- Authentication Service.
- Inventory Service.
- Alert Service.

Communication between services is achieved through REST APIs, enabling loose coupling and independent scalability.

5.1.6 Monitoring Implementation

The monitoring infrastructure is implemented using **Prometheus** and **Grafana**.

Prometheus continuously collects metrics from application services and Kubernetes resources, while Grafana provides real-time dashboards displaying:

- CPU usage.
- Memory utilization.
- HTTP request rate.
- Response time.
- Service availability.
- Inventory metrics.
- Alert statistics.

This monitoring stack enables proactive system observation and performance analysis.

5.1.7 Infrastructure as Code

Infrastructure provisioning and configuration are automated using **Terraform** and **Ansible**.

Terraform provisions the Azure resources required for deployment, while Ansible automates server configuration by installing Docker, K3s, Nginx, and monitoring tools.

This approach ensures consistent and repeatable infrastructure deployment.

5.1.8 Containerization

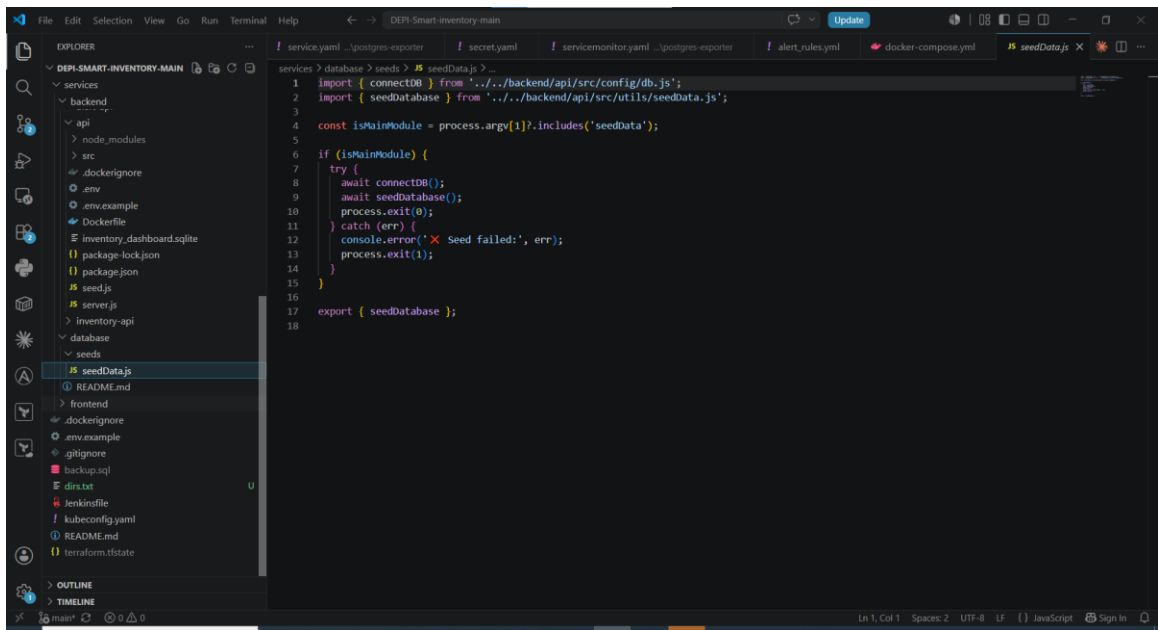
All application components are containerized using **Docker**.

Each container includes its required dependencies, ensuring consistent execution across development, testing, and production environments.

Containerized services include:

- React Frontend.
- Inventory API.

- Alert Service.
- PostgreSQL.
- Prometheus.
- Grafana.
- Nginx.



The containers are orchestrated using **K3s Kubernetes**, providing automated deployment, scaling, and fault recovery.

5.1.9 Security & Error Handling

The implementation incorporates several security mechanisms and error-handling strategies to improve system reliability.

Security measures include:

- User authentication.
- Input validation.
- Protected REST APIs.
- Environment variable configuration.
- Secure database communication.

Error handling includes:

- HTTP status codes.
- Structured exception handling.
- API validation.
- Logging.
- Database error management.

These mechanisms improve application stability and simplify debugging.

5.1.10 Coding Standards & Naming Conventions

The project follows consistent coding standards to improve readability and maintainability.

The adopted conventions include:

Element	Convention	Example
Variables	camelCase	productName
Functions	camelCase	getProducts()
Classes	PascalCase	InventoryService
React Components	PascalCase	Dashboard.jsx
Constants	UPPER_CASE	API_BASE_URL
API Endpoints	kebab-case	/api/inventory
Database Tables	PascalCase	Products

Additional coding practices include:

- Consistent code formatting.
- Modular design.
- Reusable functions and components.
- Meaningful variable names.
- Separation of concerns.
- Well-structured folder organization.

5.2 Version Control & Collaboration

Version control plays a vital role in modern software development by enabling efficient source code management, collaboration, and tracking of project evolution. Git and GitHub were used throughout the development process to maintain the project source code, manage changes, and automate deployment through the CI/CD pipeline.

5.2.1 GitHub Repository

The complete source code of the Smart Inventory Management System is hosted on GitHub. The repository contains the frontend, backend, infrastructure configuration, Kubernetes manifests, monitoring configuration, and project documentation.

The repository provides:

- Centralized source code management.
- Version history.
- Issue tracking.
- Documentation.
- Collaboration support.
- CI/CD integration.

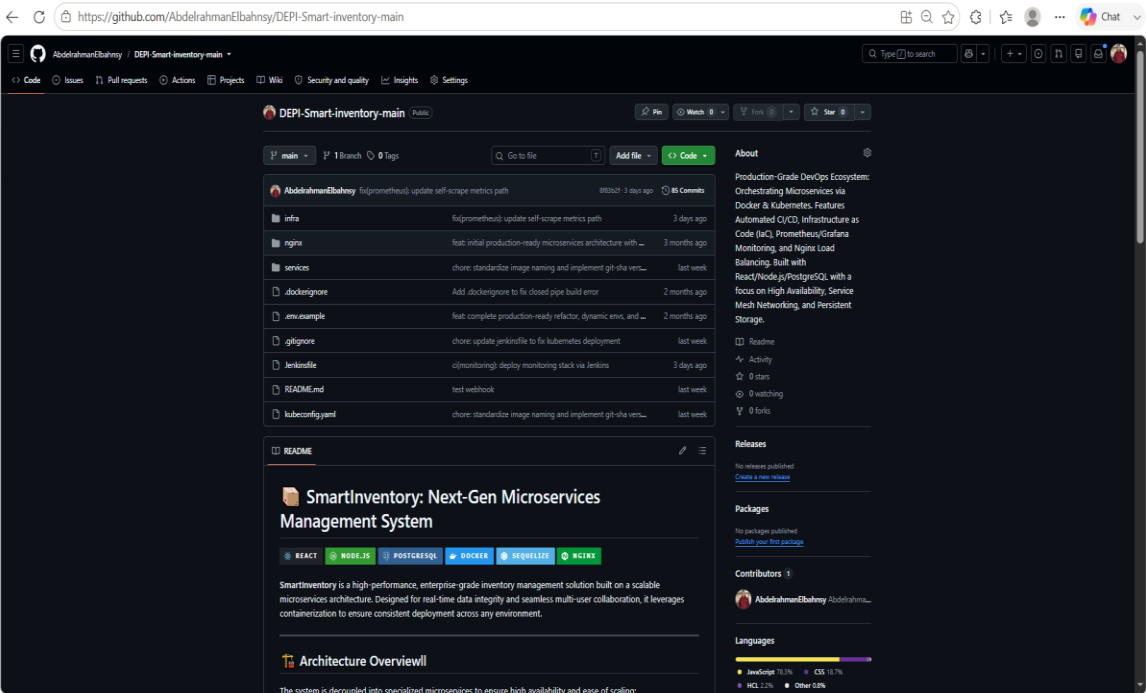
Repository Link:

(<https://github.com/AbdelrahmanElbahnsy/DEPI-Smart-inventory-main>)

5.2.2 Repository Structure

The repository is organized into multiple directories to separate application components and infrastructure resources.

```
Smart-Inventory-System/  
├── frontend/  
├── backend/  
├── kubernetes/  
├── terraform/  
├── ansible/  
├── monitoring/  
│   ├── prometheus/  
│   └── grafana/  
├── nginx/  
├── jenkins/  
├── docs/  
├── docker-compose.yml  
├── README.md  
└── LICENSE
```



This modular structure improves maintainability and simplifies project navigation.

5.2.3 Branching Strategy

The project follows a simplified **Feature Branch Workflow** to organize development activities.

The following branches are used:

Branch	Purpose
main	Stable production-ready code
develop	Integration branch for ongoing development
feature/*	Development of new features
hotfix/*	Emergency production fixes

Development workflow:

1. Create a feature branch.
2. Implement the required functionality.
3. Commit changes with meaningful messages.

4. Merge into the development branch.
5. Test the application.
6. Merge into the main branch after successful validation.

5.2.4 Commit History

Meaningful commit messages are used to document project progress and simplify version tracking.

Examples include:

Initial project setup

Implement inventory CRUD operations

Add alert service

Configure Docker containers

Deploy application using K3s

Integrate Prometheus monitoring

Configure Grafana dashboards

Implement Jenkins CI/CD pipeline

Fix authentication issues

Update Kubernetes manifests

Each commit represents a logical development step, making it easier to review changes and identify issues.

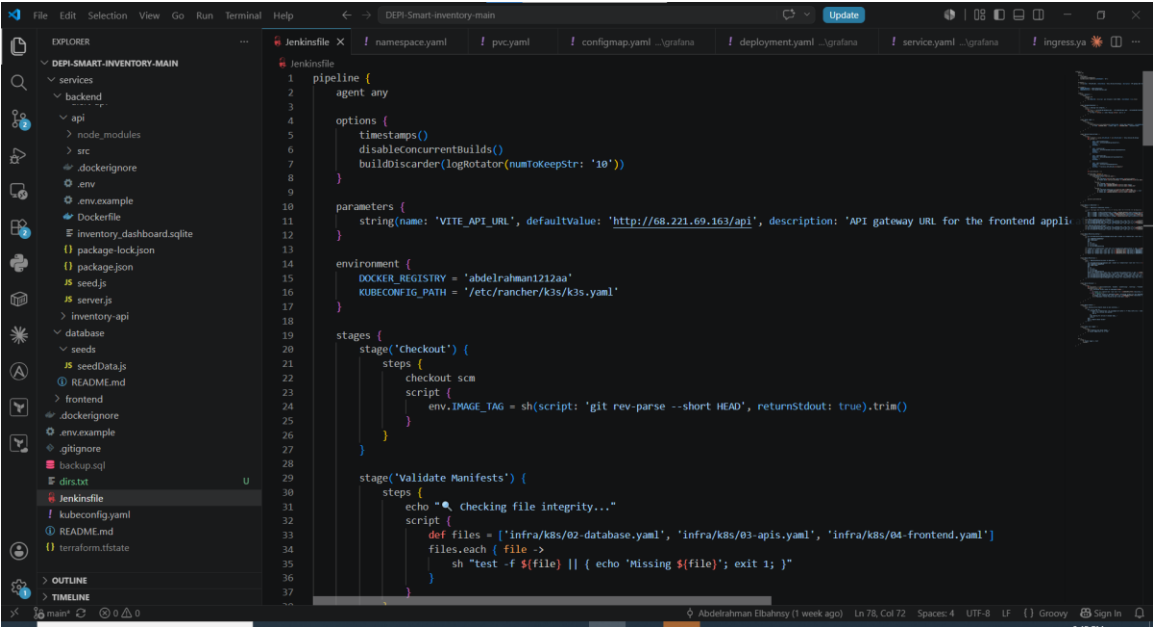
5.2.5 CI/CD Integration

Continuous Integration and Continuous Deployment (CI/CD) are implemented using **Jenkins**.

The CI/CD pipeline performs the following tasks automatically:

- Clone the GitHub repository.
- Install project dependencies.
- Build application components.
- Build Docker images.
- Deploy containers to the K3s cluster.
- Verify deployment status.
- Restart updated services if required.

This automation reduces manual deployment effort and improves software delivery reliability.



5.3 Deployment & Execution

This section describes the deployment process and execution environment required to run the Smart Inventory Management System successfully.

5.3.1 System Requirements

Hardware Requirements

Component	Minimum Requirement
-----------	---------------------

Processor	Dual-Core CPU
-----------	---------------

Memory	8 GB RAM
--------	----------

Storage	30 GB Available Disk
---------	----------------------

Network	Stable Internet Connection
---------	----------------------------

Software Requirements

- Ubuntu Linux
- Docker
- K3s Kubernetes
- Git
- Jenkins
- Terraform
- Ansible
- PostgreSQL
- Prometheus
- Grafana
- Nginx
- Node.js
- npm

5.3.2 Installation Steps

The following steps are required to install the project:

1. Clone the GitHub repository.
2. Install project dependencies.
3. Configure environment variables.

4. Build Docker images.
5. Deploy Kubernetes manifests.
6. Verify running services.
7. Access the web application.

5.3.3 Environment Configuration

The application configuration is managed through environment variables.

Typical configuration includes:

- Database connection string.
- API URLs.
- JWT Secret.
- Grafana configuration.
- Prometheus targets.
- Nginx configuration.

Sensitive information is stored securely and excluded from version control.

5.3.4 Running the Application

The application can be executed locally or on the deployment server.

Execution process:

1. Start Docker services.
2. Start the K3s cluster.
3. Deploy Kubernetes manifests.
4. Verify pod status.
5. Access the frontend through the configured URL.

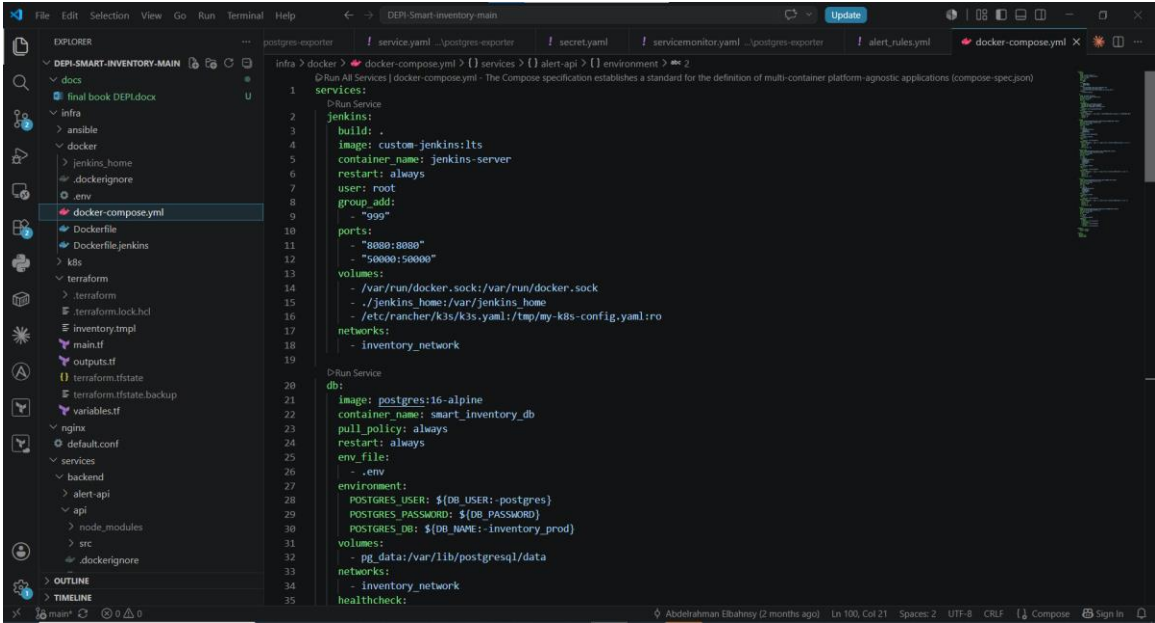
5.3.5 Docker Deployment

Each application component is packaged as an independent Docker image.

Deployment steps:

- Build Docker images.
- Create application containers.
- Configure networking.
- Verify container health.
- Push images when required.

Docker ensures consistent execution across all environments.



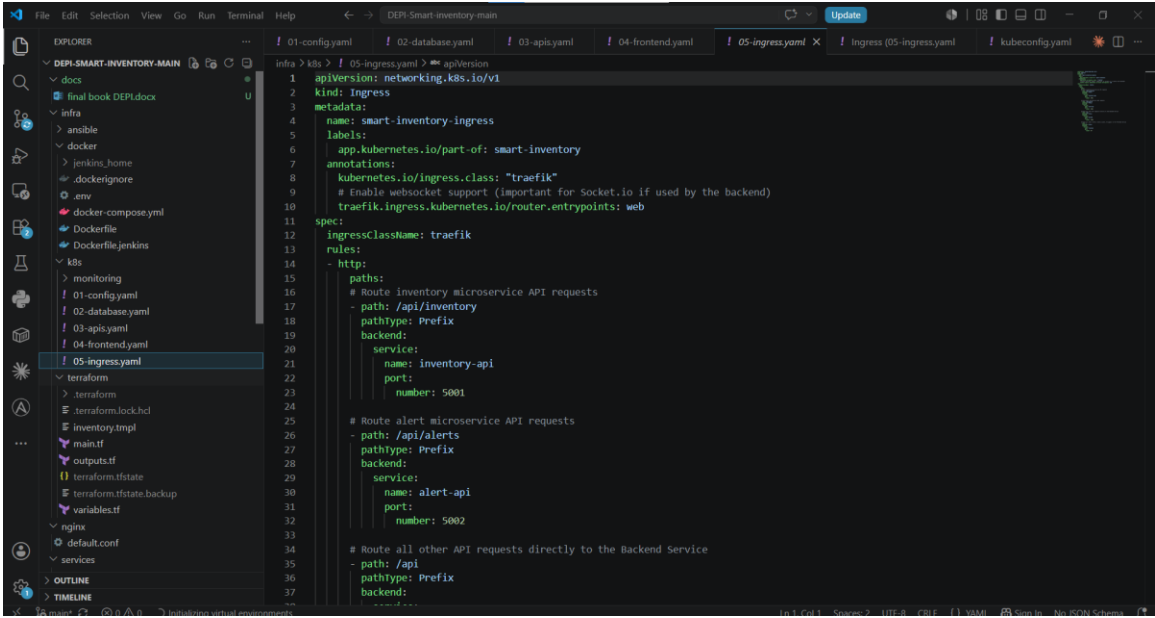
5.3.6 Kubernetes Deployment

The application is deployed on a **K3s Kubernetes** cluster.

Deployment resources include:

- Deployments
- Pods
- Services
- ConfigMaps
- Secrets
- Persistent Volumes
- Nginx Ingress

Kubernetes automatically manages container scheduling, recovery, and scaling.



5.3.7 Jenkins Deployment Pipeline

The deployment pipeline automates the software delivery process.

Pipeline workflow:

GitHub



Jenkins



Build

↓

Test

↓

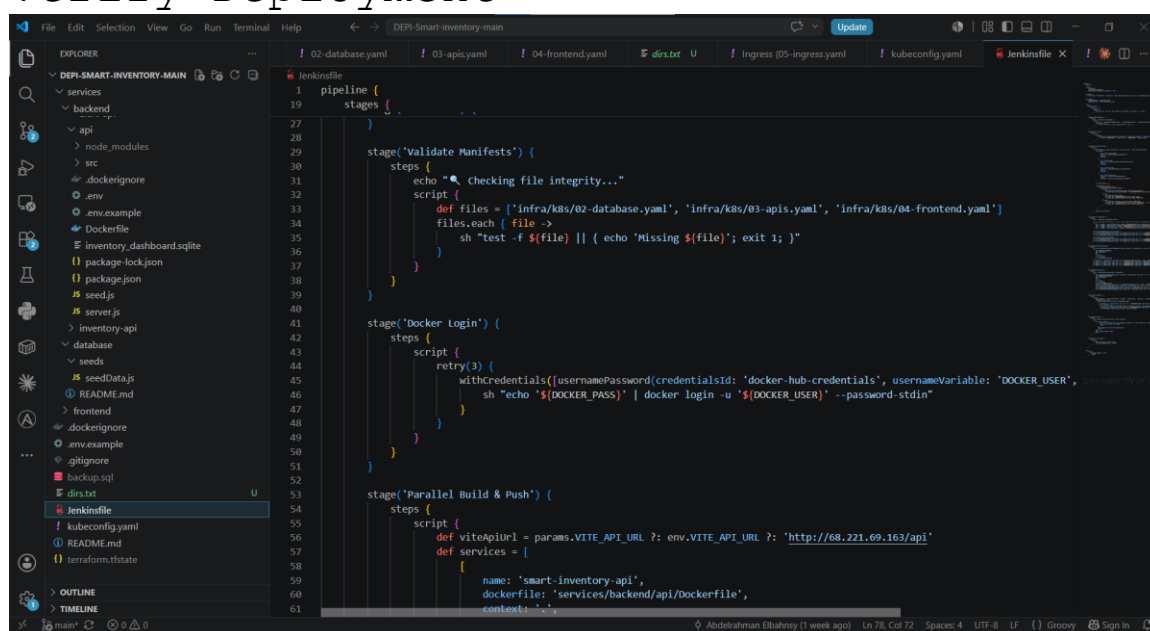
Docker Build

↓

Deploy to K3s

↓

Verify Deployment



This process enables continuous deployment with minimal manual intervention.

5.3.8 Monitoring Execution

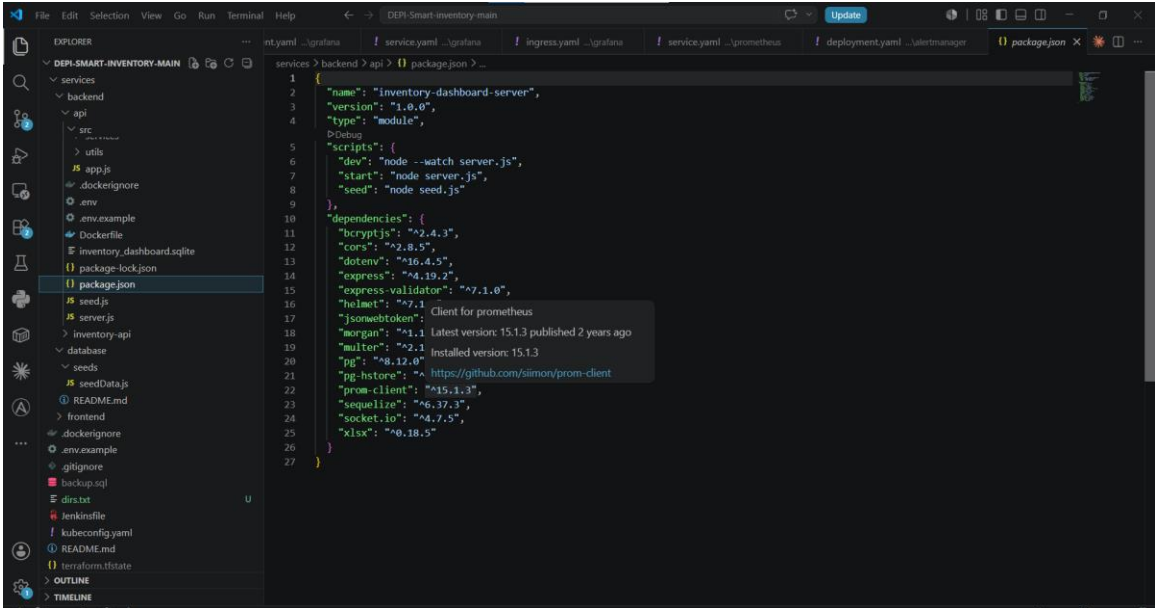
After deployment, monitoring services are started automatically.

The monitoring stack consists of:

- Prometheus
- Grafana
- Node Exporter

The monitoring dashboards display:

- CPU utilization
- Memory usage
- HTTP request rate
- Response time
- Active requests
- Kubernetes status
- Inventory metrics



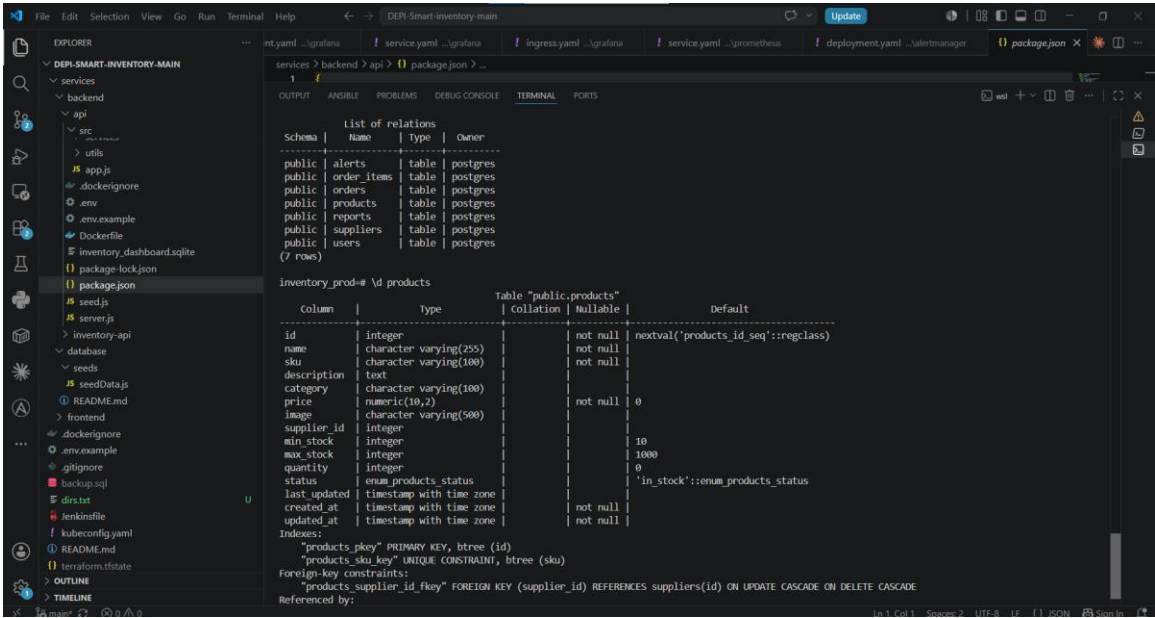
5.4.2 Inventory Management Code

The inventory module implements the core business logic for managing products and stock levels.

Implemented operations include:

- Add Product
- Update Product
- Delete Product
- Retrieve Products
- Inventory Search
- Stock Update

Figure 5.2: Inventory Service Source Code



5.4.3 Alert Service Code

The Alert Service continuously checks inventory quantities and generates alerts whenever product stock falls below the configured threshold.

Main functions include:

- Threshold Verification
- Alert Generation
- Notification Processing
- Alert Status Update

Figure 5.3: Alert Service Implementation

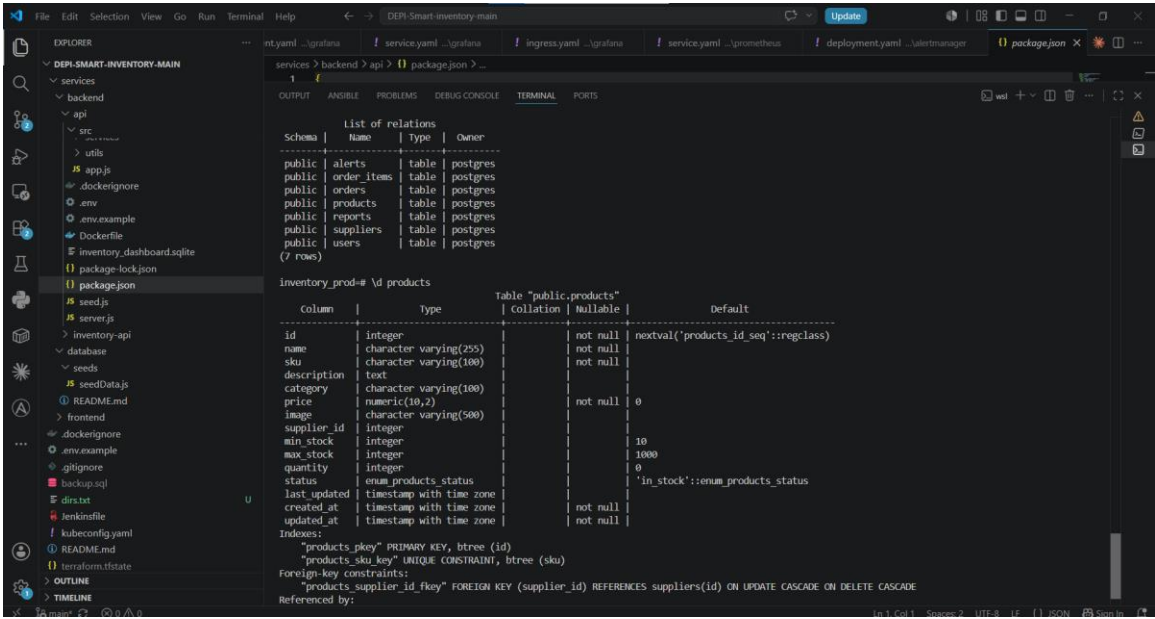
5.4.4 Database Connection Code

Database connectivity is implemented using PostgreSQL.

The implementation includes:

- Database Configuration
- Connection Pool
- Query Execution
- Error Handling
- Transaction Management

Figure 5.4: PostgreSQL Connection Code



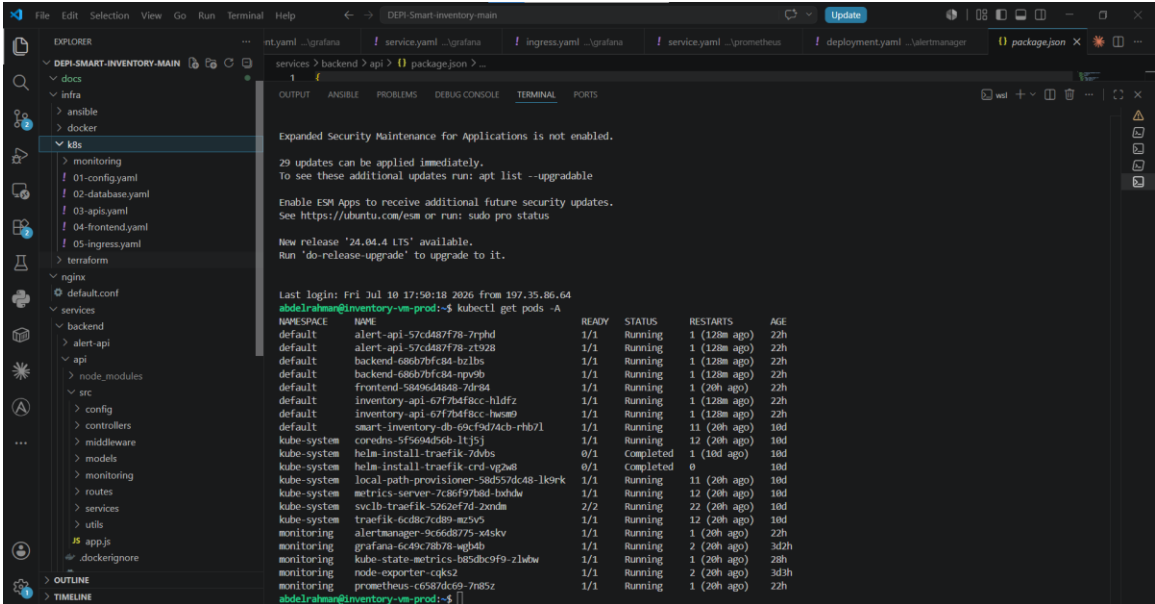
5.4.5 Kubernetes Manifests

Kubernetes manifest files define how the application is deployed and managed within the K3s cluster.

Configuration files include:

- Deployments
- Services
- ConfigMaps
- Secrets
- Persistent Volumes
- Ingress

Figure 5.5: Kubernetes Deployment Manifest



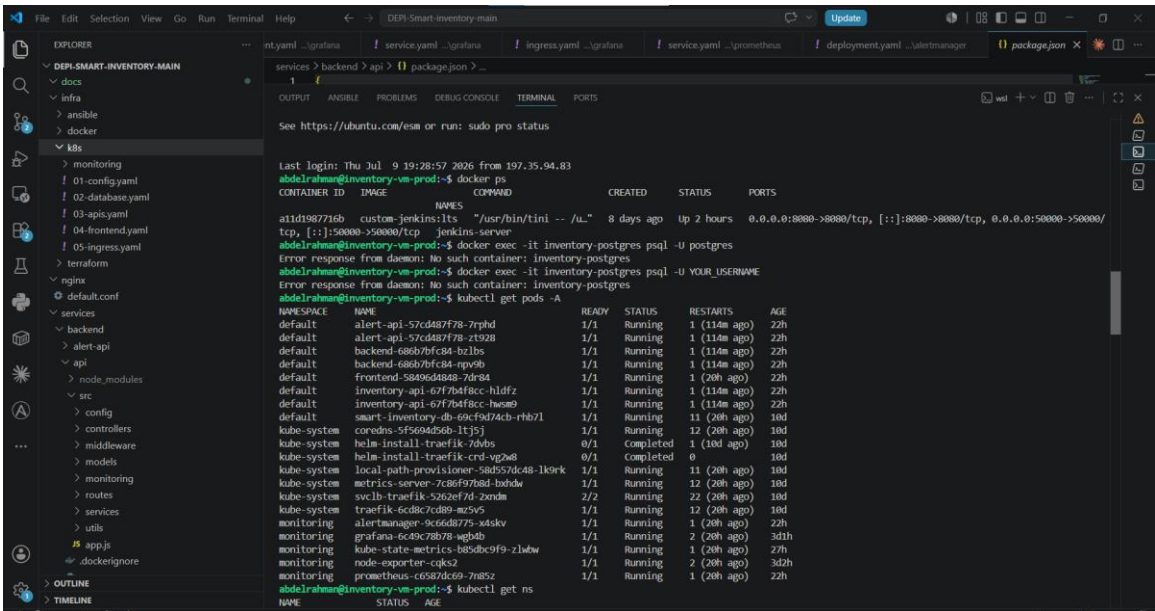
5.4.6 Jenkins Pipeline (Jenkinsfile)

The Jenkins pipeline automates the build, test, containerization, and deployment processes.

Pipeline stages include:

- Checkout Source Code
- Install Dependencies
- Build Application
- Docker Image Build
- Kubernetes Deployment
- Deployment Verification

Figure 5.6: Jenkins Pipeline (Jenkinsfile)



5.4.7 Terraform Configuration

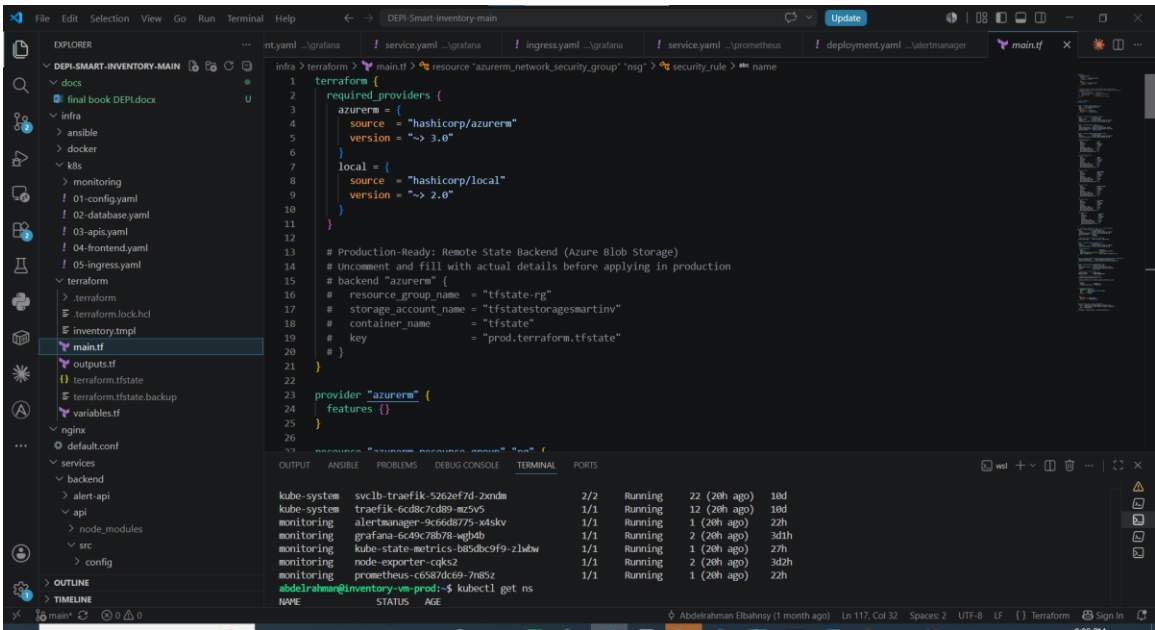
Terraform is used to provision the cloud infrastructure required for the project.

Resources include:

- Azure Virtual Machine
- Virtual Network
- Storage Resources
- Security Configuration

Infrastructure is managed using Infrastructure as Code (IaC), ensuring repeatable and consistent deployments.

Figure 5.7: Terraform Configuration



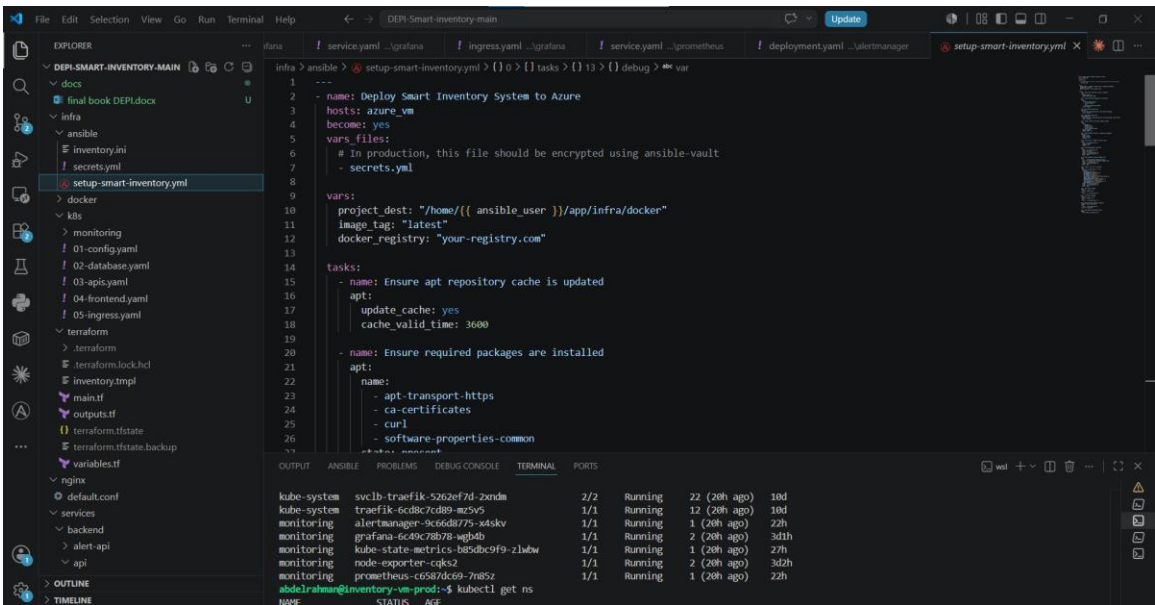
5.4.8 Ansible Playbooks

Ansible automates the server configuration and application deployment process.

Playbooks perform:

- Docker Installation
- K3s Installation
- Nginx Configuration
- Monitoring Deployment
- Application Deployment

Figure 5.8: Ansible Playbook



5.5 README & User Guide

The project includes comprehensive documentation to simplify installation, configuration, execution, and maintenance. The README file serves as the primary reference for developers and users.

5.5.1 Installation Guide

The installation process consists of the following steps:

1. Clone the project repository.
2. Install the required dependencies.
3. Configure environment variables.
4. Build Docker images.
5. Deploy the Kubernetes resources.
6. Verify that all services are running.
7. Access the application through the configured URL.

5.5.2 Configuration Guide

Before running the application, the following configuration parameters should be defined:

- Database Connection String
- API Base URL
- JWT Secret Key
- Prometheus Configuration
- Grafana Configuration
- Nginx Configuration
- Kubernetes Secrets
- Environment Variables

Configuration files are separated from the source code to improve security and flexibility.

5.5.3 API Usage

The system exposes RESTful APIs that enable communication between the frontend and backend services.

Main API groups include:

API	Description
Authentication API	User authentication and authorization
Inventory API	Product and inventory management
Alert API	Low-stock monitoring and alerts
Metrics API	Prometheus metrics endpoint

API requests and responses are formatted using JSON and follow REST principles.

5.5.4 Project Repository

The complete project source code is maintained using Git and hosted on GitHub.

The repository contains:

- Frontend Source Code
- Backend Source Code
- Kubernetes Manifests
- Docker Configuration
- Terraform Scripts
- Ansible Playbooks
- Monitoring Configuration
- Project Documentation

Repository URL:

<https://github.com/AbdelrahmanElbahnsy/DEPI-Smart-inventory-main>

5.5.5 Deployment Link

The deployed application can be accessed through the project deployment URL.

Deployment environment:

- Microsoft Azure Virtual Machine
- Ubuntu Linux
- Docker
- K3s Kubernetes
- Nginx Reverse Proxy
- Prometheus Monitoring
- Grafana Dashboards

Application URL:

smart-inventory-abdelrahmanelbahnsy.spaincentral.cloudapp.azure.com

Chapter 6

Testing & Quality Assurance

Testing & Quality Assurance

6.1 Test Cases & Test Plan

6.1.1 Testing Strategy

The Smart Inventory Management System was tested using multiple testing techniques to ensure functionality, reliability, performance, and deployment stability. Testing covered both application-level features and cloud infrastructure components.

The testing strategy included:

- Functional Testing
- Integration Testing
- System Testing
- Deployment Testing
- Monitoring Verification
- User Acceptance Testing (UAT)

6.1.2 Testing Environment

Testing was conducted in an environment that closely resembles the production deployment.

Component	Technology
Operating System	Ubuntu Linux
Cloud Platform	Microsoft Azure
Container Runtime	Docker
Orchestration	K3s Kubernetes
Database	PostgreSQL
Reverse Proxy	Nginx
Monitoring	Prometheus & Grafana
CI/CD	Jenkins

6.1.3 Test Plan

The testing process followed these stages:

1. Verify frontend functionality.
2. Validate backend APIs.
3. Test database operations.
4. Verify communication between microservices.
5. Deploy the application to K3s.

- 6. Verify monitoring metrics.
- 7. Validate Jenkins deployment.
- 8. Perform user acceptance testing.

6.1.4 Test Cases

Test ID	Test Case	Expected Result	Status
TC-01	User Login	User successfully logs in	Passed
TC-02	Add Product	Product added successfully	Passed
TC-03	Update Product	Product updated	Passed
TC-04	Delete Product	Product removed	Passed
TC-05	Search Product	Correct product displayed	Passed
TC-06	Generate Low Stock Alert	Alert created automatically	Passed
TC-07	Database Connection	Successful connection	Passed
TC-08	API Response	HTTP 200 returned	Passed
TC-09	Kubernetes Deployment	Pods running successfully	Passed
TC-10	Grafana Dashboard	Metrics displayed correctly	Passed
TC-11	Prometheus Targets	All targets UP	Passed
TC-12	Jenkins Pipeline	Build and deployment completed	Passed

6.1.5 Test Results

All planned test cases were executed successfully. The application met all functional requirements, and the deployment infrastructure operated without critical failures. Monitoring dashboards confirmed stable application performance throughout the testing period.

6.2 Automated Testing

6.2.1 API Testing

RESTful APIs were tested using tools such as Postman to verify endpoint availability, request validation, response accuracy, and HTTP status codes.

The following APIs were validated:

- Authentication API
- Inventory API
- Alert API

- Metrics API

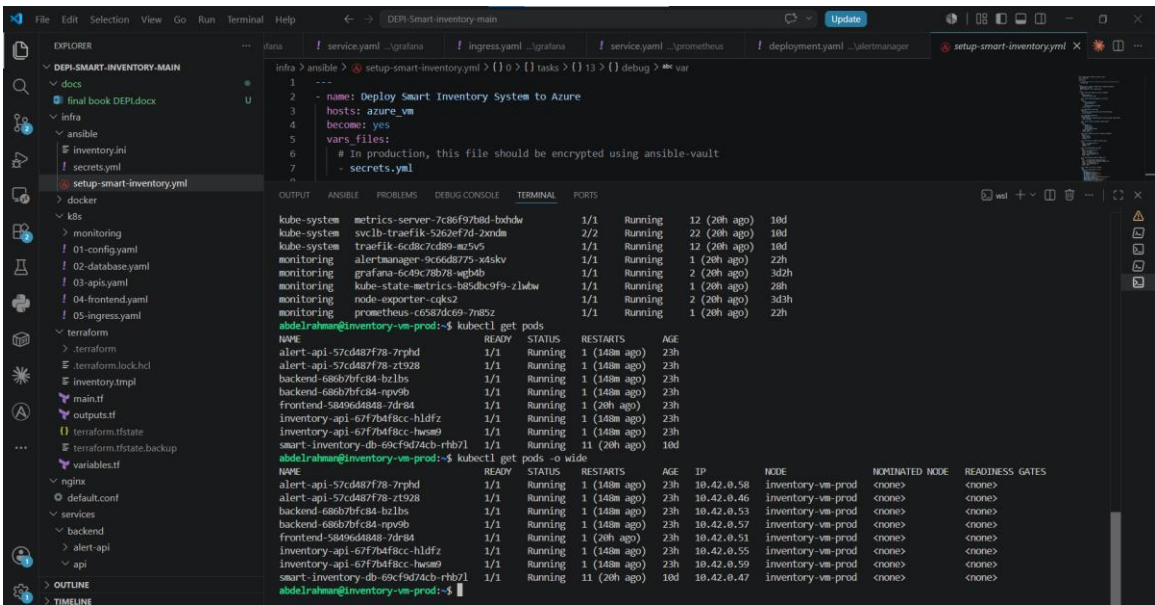
6.2.2 Kubernetes Deployment Validation

After deployment, Kubernetes resources were validated using administrative commands.

Validation included:

- Pod status
- Deployment status
- Service availability
- Ingress configuration
- Persistent Volume mounting

All Kubernetes resources were successfully deployed and remained in the **Running** state.



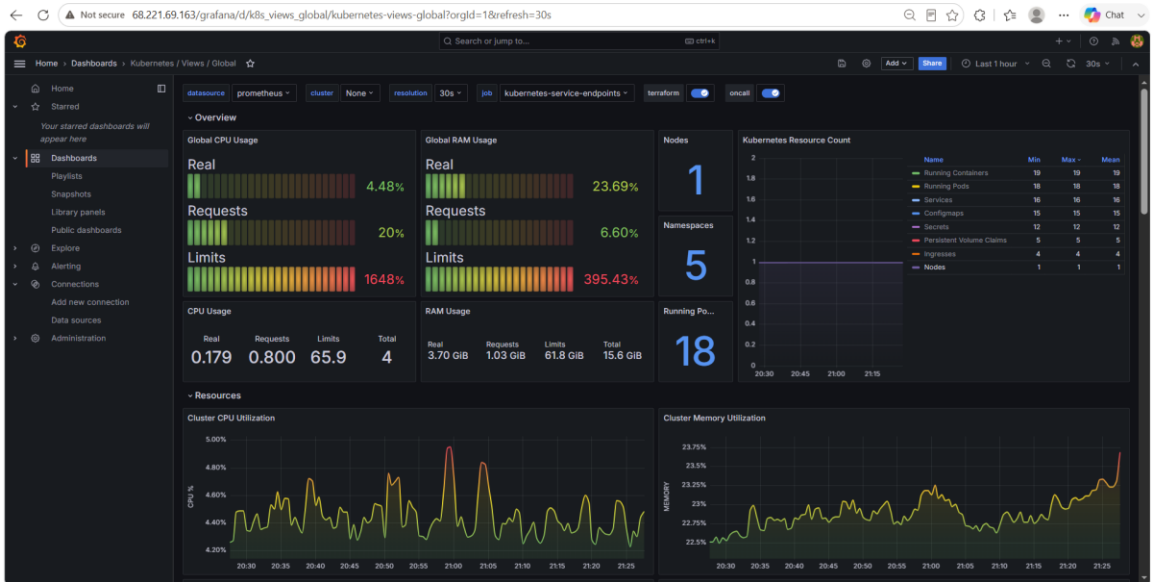
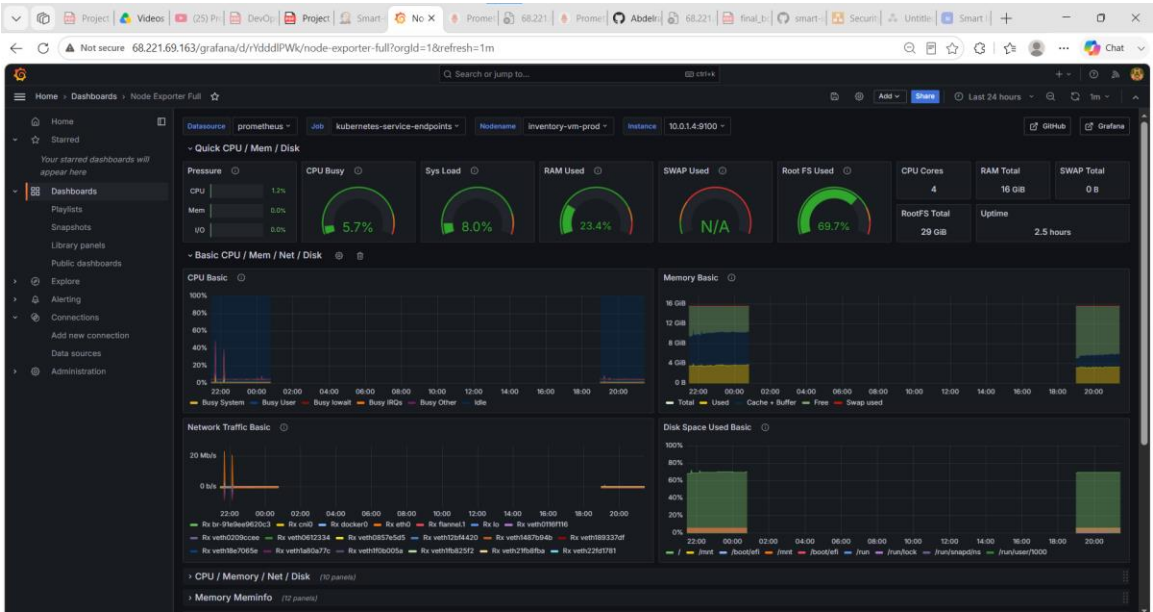
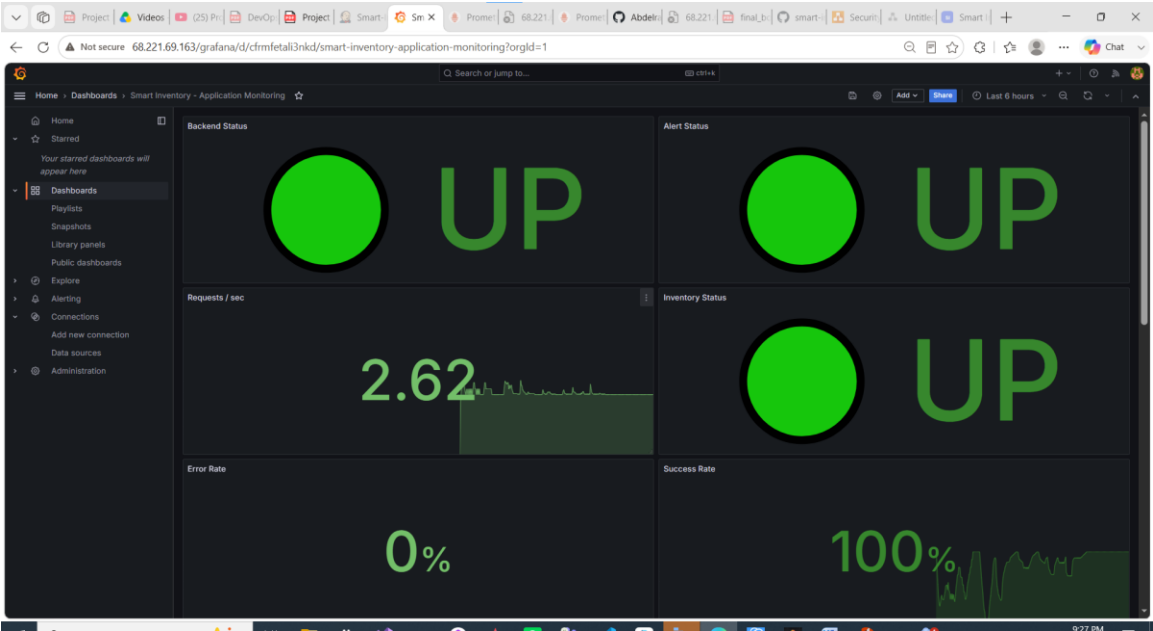
6.2.3 Monitoring Verification

Prometheus successfully collected metrics from all application services.

Grafana dashboards correctly displayed:

- Request Rate
- Response Time
- Active Requests
- Service Health
- CPU Usage
- Memory Usage

Monitoring confirmed that the deployed application remained healthy during execution



6.2.4 CI/CD Pipeline Validation

The Jenkins pipeline was executed to validate automated deployment.

The pipeline successfully completed the following stages:

- Source Code Checkout
- Dependency Installation
- Build
- Docker Image Creation
- Kubernetes Deployment
- Deployment Verification

No deployment failures were observed during testing.

6.3 Bug Reports

6.3.1 Identified Issues

During development, several issues were encountered and resolved.

Issue	Description	Resolution
Kubernetes Pods Restarting	Pod restart after deployment	Updated deployment configuration
Database Connectivity	Incorrect database connection settings	Fixed environment variables
Nginx Routing	API requests not forwarded correctly	Updated Nginx configuration
Jenkins Pipeline Failure	Missing Docker permissions	Updated Jenkins configuration
Monitoring Targets Down	Incorrect Prometheus target configuration	Updated scrape configuration

6.3.2 Root Cause Analysis

Each issue was analyzed to determine its root cause before implementing a corrective solution.

Common causes included:

- Configuration errors.
- Incorrect environment variables.
- Kubernetes manifest issues.
- Service dependency timing.
- Network routing configuration.

6.3.3 Applied Fixes

The following corrective actions were implemented:

- Updated Kubernetes manifests.
- Improved Docker configuration.
- Corrected database environment variables.
- Optimized Jenkins pipeline.
- Updated Prometheus scrape targets.
- Improved Nginx reverse proxy configuration.

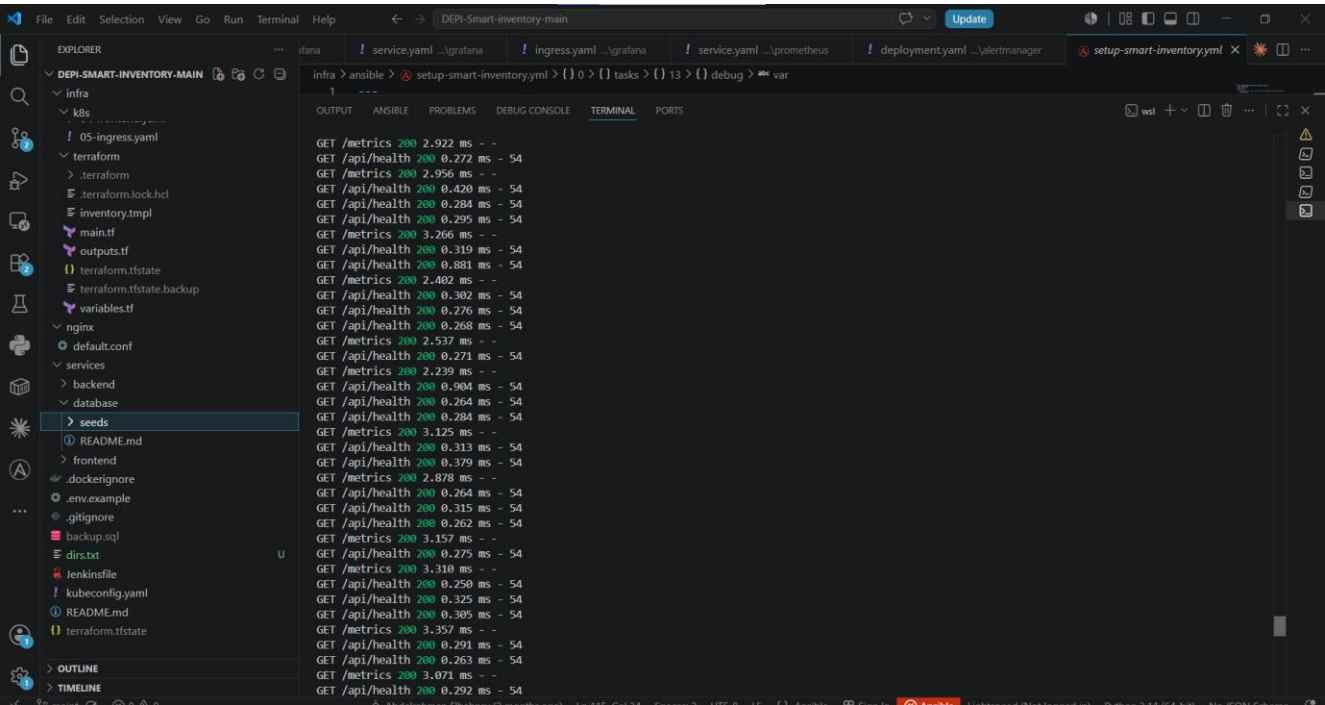
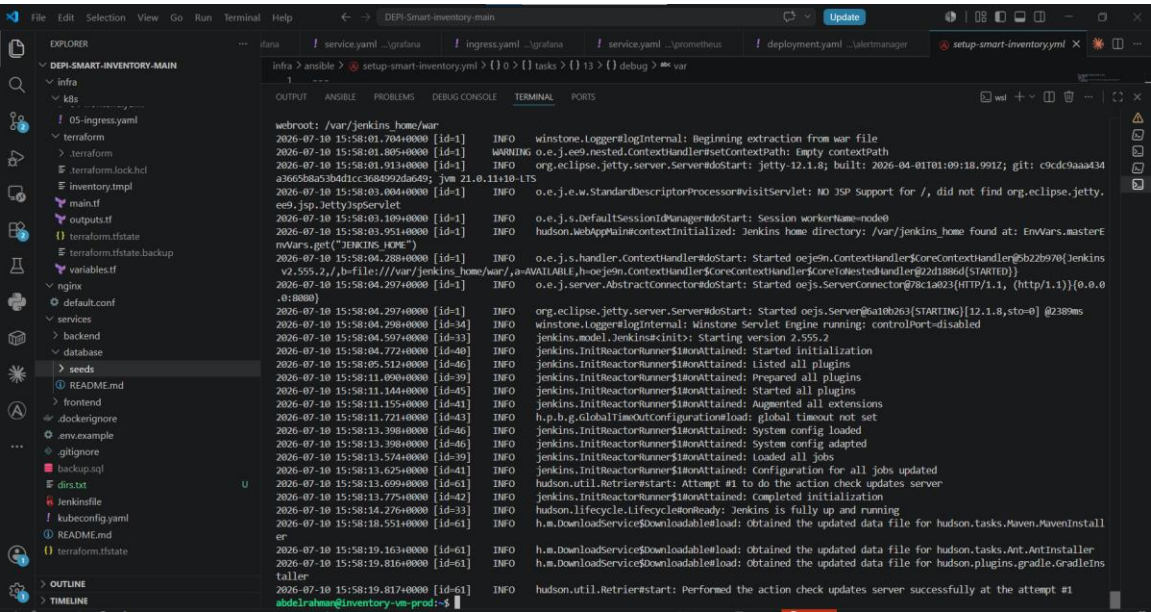
6.3.4 Final Validation

After applying all fixes, the application was redeployed and tested again.

The final validation confirmed that:

- All application services were operational.
- Kubernetes deployments were stable.
- CI/CD pipeline executed successfully.
- Monitoring dashboards displayed accurate metrics.
- REST APIs responded correctly.
- Database operations completed successfully.
- No critical issues remained.

```
abdelrahman@inventory-vm-prod:~$ kubectl logs alert-api-57cd487f78-7rphd
Defaulted container "alert-api" out of: alert-api, wait-for-db (init)
🔔 Alert API – DB Connection attempt: smart-inventory-db:5432 [inventory_prod] (retries left: 4)
✅ Alert API – PostgreSQL connected successfully
✅ Alert API – Tables synced successfully
🔔 Alert Scanner started – checking every 5 minutes
✅ Alert API running on port 5002
```



```
2026-07-10 15:58:19.817+0000 [id=61] INFO hudson.util.Retrier#start: Perfor
U
abdelrahman@inventory-vm-prod:~$ docker images

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
custom-jenkins:lts 03210e5d4947 890MB 316MB U
abdelrahman@inventory-vm-prod:~$ docker network ls
NETWORK ID NAME DRIVER SCOPE
aba5fd79380f bridge bridge local
91e9ee9620c3 docker_inventory_network bridge local
de2eb8fc1189 host host local
1e5ee8626252 none null local
abdelrahman@inventory-vm-prod:~$ kubectl get events
```

```
2026-07-10 15:58:19.817+0000 [id=61] INFO hudson.util.Retrier#start: Performed the ac
abdelrahman@inventory-vm-prod:~$ docker images

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
custom-jenkins:lts 03210e5d4947 890MB 316MB U
abdelrahman@inventory-vm-prod:~$ docker network ls
NETWORK ID NAME DRIVER SCOPE
aba5fd79380f bridge bridge local
91e9ee9620c3 docker_inventory_network bridge local
de2eb8fc1189 host host local
1e5ee8626252 none null local
abdelrahman@inventory-vm-prod:~$ kubectl get events
No resources found in default namespace.
abdelrahman@inventory-vm-prod:~$ kubectl get pods | grep nginx
abdelrahman@inventory-vm-prod:~$ kubectl get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
alert-api ClusterIP 10.43.85.15 <none> 5002/TCP 10d
backend ClusterIP 10.43.128.82 <none> 5000/TCP 10d
frontend ClusterIP 10.43.144.253 <none> 80/TCP 10d
inventory-api ClusterIP 10.43.119.74 <none> 5001/TCP 10d
kubernetes ClusterIP 10.43.0.1 <none> 443/TCP 10d
smart-inventory-db ClusterIP 10.43.248.219 <none> 5432/TCP 10d
abdelrahman@inventory-vm-prod:~$ kubectl get ingress
NAME CLASS HOSTS ADDRESS PORTS AGE
smart-inventory-ingress traefik * 10.0.1.4 80 10d
abdelrahman@inventory-vm-prod:~$ kubectl get configmap
NAME DATA AGE
inventory-config 6 10d
kube-root-ca.crt 1 10d
abdelrahman@inventory-vm-prod:~$ kubectl get secrets
NAME TYPE DATA AGE
inventory-secrets Opaque 3 10d
abdelrahman@inventory-vm-prod:~$
```

```
abdelrahman@inventory-vm-prod:~$ kubectl top nodes
NAME CPU(cores) CPU(%) MEMORY(bytes) MEMORY(%)
inventory-vm-prod 183m 4% 4411Mi 27%
abdelrahman@inventory-vm-prod:~$
```

Abdelrahman Elbahnsy (2 months ago) Ln 115, Col 34 Spaces: 2 UTF-8 LF { } Ansi

